

6.1 引言 6.2 汇编语言 6.3 编程 6.4 机器语言 6.5 灯光、摄像、开拍：编译、汇编与加载* 6.6 杂项* 6.7 RISC-V架构的演进 6.8 另一视角：x86架构 6.9 总结 习题 面试问题

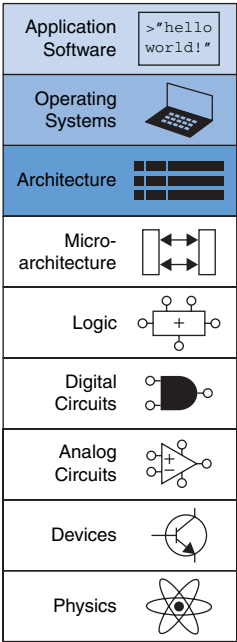
6.1 引言

前几章介绍了数字设计原理和构建模块。本章将提升几个抽象层次，定义计算机的体系结构。*architecture*是程序员对计算机的视角，由指令集（语言）和操作数位置（寄存器和内存）定义。存在多种不同的体系结构，例如RISC-V、ARM、x86、MIPS、SPARC和PowerPC。

理解任何计算机体系结构的第一步是学习其语言。计算机语言中的单词称为*instructions*。计算机的词汇表称为*instruction set*。所有在计算机上运行的程序都使用相同的指令集。即使是复杂的软件应用程序，如文字处理和电子表格应用程序，最终也会被编译成一系列简单的指令，例如加法、减法和分支。计算机指令既指示要执行的操作，也指示要使用的操作数。操作数可能来自内存、寄存器或指令本身。

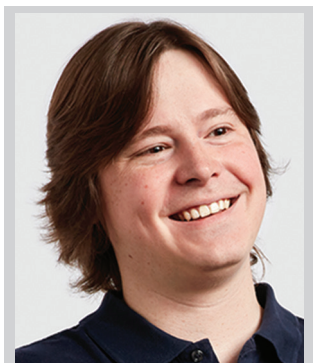
计算机硬件只理解1和0，因此指令以称为*machine language*的格式编码为二进制数。正如我们用字母编码人类语言一样，计算机用二进制数编码机器语言。RISC-V架构将每条指令表示为一个32位的字。微处理器是读取并执行机器语言指令的数字系统。然而，人类认为阅读机器语言很繁琐，因此我们更倾向于用称为*assembly language*的符号格式来表示指令。

不同架构的指令集更像是不同的方言而非不同的语言。几乎所有架构都定义了基本指令——例如加法、减法和分支——这些指令对内存或寄存器进行操作。一旦你学会了一种指令集，理解其他指令集就相当直接了。





Krste Asanovic 在暑期项目中启动了 RISC-V。他是加州大学伯克利分校的计算机科学教授，也是 RISC-V 国际（原 RISC-V 基金会）的董事会主席。他还是 SiFive 的联合创始人，该公司致力于开发并商业化 RISC-V 芯片、开发板及配套工具。（照片经许可使用。）



Andrew Waterman 在 SiFive 设计微处理器，这家公司是他与 Krste Asanovic 于 2015 年共同创立的，旨在提供低成本的 RISC-V 核心和定制芯片。他于 2016 年在加州大学伯克利分校获得计算机科学博士学位，在那里，由于厌倦了现有指令集架构的反复无常，他共同设计了 RISC-V ISA 以及首批 RISC-V 核心。（照片经许可使用。）

计算机架构并不定义底层硬件实现。通常，同一架构存在多种不同的硬件实现。例如，英特尔和超威半导体（AMD）都销售属于相同 x86 架构的各种微处理器。这些微处理器都能运行相同的程序，但采用不同的底层硬件。因此，它们在性能、价格和功耗方面各有取舍。有些微处理器针对高性能服务器进行了优化，而另一些则针对设备或笔记本电脑的长电池续航进行了优化。寄存器、存储器、算术逻辑单元（ALU）及其他构建模块的具体排列方式被称为 *microarchitecture*，这将是第 7 章的主题。

本文介绍 RISC-V（发音为“risk five”）架构，这是首个获得广泛商业支持的开源指令集架构。我们描述 RISC-V 32 位整数指令集（RV32I）2.2 版本，该版本构成了 RISC-V 指令集的核心。第 6.6 节和第 6.7 节总结了该架构其他版本的特点。在线提供的 *RISC-V Instruction Set Manual* 是架构的权威定义。

RISC-V 架构最初于 2010 年由 Krste Asanovic、Andrew Waterman、David Patterson 等人在加州大学伯克利分校定义。自诞生以来，许多人对其发展做出了贡献。RISC-V 的独特之处在于其开源特性使其免费使用，并且在能力上与 ARM 和 x86 等商业架构相当。迄今为止，只有少数公司（包括 SiFive 和 Western Digital）制造了商业芯片，但其采用率正在迅速增长。我们通过介绍汇编语言指令、操作数位置以及常见编程结构（如分支、循环、数组操作和函数调用）来开启对 RISC-V 架构的理解之旅。然后，我们描述汇编语言如何转换为机器语言，并展示程序如何加载到内存中并执行。

在本章中，我们描述了 RISC-V 架构的设计如何受到 David Patterson 和 John Hennessy 在其著作 *Computer Organization and Design* 中阐述的四项原则的驱动：(1) 规律性支持简洁性；(2) 让常见情况更快；(3) 越小越快；(4) 好的设计需要好的权衡。

6.2 汇编语言

Assembly language 是计算机原生语言的人类可读表示。每条汇编语言指令既指定了要执行的操作，也指定了操作所针对的操作数。我们介绍简单的算术指令，并展示这些操作如何

是用汇编语言编写的。然后我们定义RISC-V指令的操作数：寄存器、内存和常量。

本章假设您已经对诸如C、C++或Java等高级编程语言有一定了解。（对于本章中的大多数示例，这些语言实际上几乎相同，但在有差异的地方，我们将使用C语言。）附录C为那些几乎没有或没有编程经验的读者提供了C语言的入门介绍。

6.2.1 指令

计算机执行的最常见操作之一是加法。代码示例6.1展示了将变量b和c相加并将结果写入a的代码。左侧以高级语言（使用C、C++和Java的语法）显示代码，右侧则用RISC-V汇编语言重写。请注意，C程序中的语句以分号结尾。

代码示例6.1 加法

High-Level Code	RISC-V Assembly Code
a = b + c;	add a, b, c

汇编指令的第一部分，add，被称为*mnemonic*，指示要执行的操作。该操作对b和c（即*source operands*）执行，并将结果写入a（即*destination operand*）。

代码示例6.2展示了减法与加法的相似性。`sub`指令的格式与`add`指令相同：目标操作数后跟两个源操作数。这种一致的指令格式体现了第一条设计原则：

设计原则1：规律性支持简洁性。

代码示例6.2 减法

High-Level Code	RISC-V Assembly Code
a = b - c;	sub a, b, c

操作数数量一致的指令——在此例中为两个源操作数和一个目标操作数——在硬件中更易于编码和处理。更复杂的高级代码会转换为多条RISC-V指令，如代码示例6.3所示。



大卫·帕特森自1976年起担任加州大学伯克利分校计算机科学教授，并于1984年与约翰·亨尼西共同发明了*reduced instruction set computing*。该技术后来商业化成为SPARC架构。他协助开发了RISC-V架构，并持续在其发展中发挥核心作用。（照片经许可使用。）

助记符（发音为 nuh-maa-nik）源自希腊语单词 μνημονεύειν, *to remember*。汇编语言助记符比表示相同操作的机器语言中的0和1模式更容易记忆。

RISC-V被称为“五”，因为它是伯克利开发的第五个RISC架构。

在序言中，我们介绍了几个用于编译和模拟C及RISC-V汇编代码的模拟器与工具。我们还提供了实验（可参见本教材配套网站，详见序言），展示如何使用这些工具。



约翰·轩尼诗是斯坦福大学电气工程与计算机科学教授，并于2000年至2016年担任斯坦福大学校长，*reduced instruction set computing*。他还开发了MIPS计算机架构，并于1984年共同创立了MIPS计算机系统公司。MIPS处理器被广泛应用于许多商业系统中，包括硅谷图形公司、任天堂和思科的产品。约翰·轩尼诗和大卫·帕特森因开创了计算机架构设计与评估的量化方法，于2017年获得图灵奖。（照片经许可使用。）

代码示例6.3 更复杂的代码

High-Level Code	RISC-V Assembly Code
<pre>a = b + c - d; // single-line comment /* multiple-line comment */</pre>	<pre>add t, b, c # t = b + c sub a, t, d # a = t - d</pre>

在高级语言示例中，单行注释以 `//` 开头并持续到行尾，多行注释以 `/*` 开头并以 `*/` 结尾。在RISC-V汇编语言中，仅使用单行注释，以井号（`#`）开头并持续到行尾。代码示例6.3中的汇编语言程序将中间结果（`b + c`）存储在临时变量`t`中。使用多条汇编语言指令执行更复杂的操作，是计算机架构第二设计原则的一个实例：

设计原则2：让常见情况快速执行。

RISC-V指令集通过仅包含简单、常用的指令来加速常见情况。指令数量保持较少，以便解码指令及其操作数所需的硬件可以简单、小巧且快速。较少见的更复杂操作则通过多个简单指令序列来执行。因此，RISC-V是一种*reduced instruction set computer*（精简指令集计算机，RISC）架构。而拥有许多复杂指令的架构，如英特尔的x86架构，则是*complex instruction set computers*（复杂指令集计算机，CISC）。例如，x86定义了一条“字符串移动”指令，用于将字符串（一系列字符）从内存的一部分复制到另一部分。在RISC机器中，这样的操作需要许多（甚至可能数百条）简单指令。然而，在CISC架构中实现复杂指令的代价是增加硬件和开销，这会拖慢简单指令的速度。

RISC架构（如RISC-V）通过保持不同指令集的小规模，最小化了硬件复杂性和必要的指令编码。例如，一个包含64条简单指令的指令集需要 $\log_2 64 = 6$ 位来编码操作，而一个包含256条指令的指令集则需要每条指令 $\log_2 256 = 8$ 位的编码。在CISC机器中，即使复杂指令可能很少使用，它们也会给所有指令（包括简单指令）增加开销。

6.2.2 操作数：寄存器、内存和常量

指令对*operands*进行操作。在代码示例6.2中，变量`a`、`b`和`c`都是操作数。但计算机处理的是1和0，而不是变量名。指令需要一个物理位置来从中

检索二进制数据。操作数可以存储在寄存器或内存中，也可能是存储在指令本身的常量。计算机使用不同的位置来保存操作数，以优化速度和数据容量。作为常量或存储在寄存器中的操作数访问速度快，但只能保存少量数据。额外的数据必须从内存中访问，内存容量大但速度慢。RISC-V被称为32位架构，因为它处理32位数据。

寄存器
指令需要快速访问操作数以便高速运行，但存储在内存中的操作数检索耗时较长。因此，大多数架构会指定少量寄存器来存放常用操作数。RISC-V架构拥有32个寄存器，称为*register set*，存储在一个名为*register file*的小型多端口存储器中。寄存器数量越少，访问速度越快。这引出了第三条设计原则：

设计原则3：越小越快。

从桌上几本相关书籍中查找信息，比在图书馆书架上搜索信息快得多。同样，从小型寄存器文件中读取数据，也比从大型内存中读取数据更快。寄存器文件通常由小型SRAM阵列构建（参见第5.5.3节）。

代码示例6.4展示了带有寄存器操作数的加法指令。变量a、b和c被任意放置在s0、s1和s2中。名称s1读作“寄存器s1”或简称为“s1”。该指令将s1（b）和s2（c）中包含的32位值相加，并将32位结果写入s0（a）。代码示例6.5展示了使用寄存器t0存储中间计算结果b + c的RISC-V汇编代码。

RISC-V架构也存在64位和128位版本，但本书将重点介绍32位版本。更宽的版本（RV64I和RV128I）与32位版本（RV32I）几乎相同，只是寄存器和内存地址的宽度不同。其他主要新增内容包括仅对字的下半部分进行操作的指令，以及传输更宽字的内存操作。

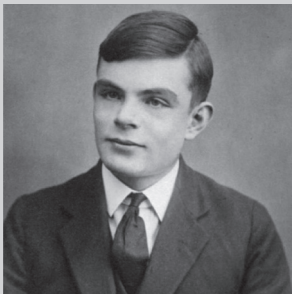
附录B位于教科书的内封面上，提供了整个RISC-V指令集的便捷总结。

代码示例6.4 寄存器操作数

High-Level Code	RISC-V Assembly Code
a = b + c;	# s0 = a, s1 = b, s2 = c add s0, s1, s2 # a = b + c

代码示例6.5 临时寄存器

High-Level Code	RISC-V Assembly Code
a = b + c - d;	# s0 = a, s1 = b, s2 = c, s3 = d, t0 = t add t0, s1, s2 # t = b + c sub s0, t0, s3 # a = t - d



艾伦·图灵，1912–1954一位英国数学家与计算机科学家，被誉为理论计算机科学与人工智能的奠基人。他发明了图灵机——一种代表抽象处理器的计算数学模型。二战期间，他研制出用于破解加密信息的机电设备，此举缩短了战争进程并拯救了数百万生命。1952年，图灵因同性性行为被起诉，为免于监禁而接受化学阉割治疗。两年后，他死于氰化物中毒。以他命名的图灵奖是计算机领域的最高荣誉，自1966年起每年颁发，目前附带100万美元奖金。

立即数可以用十进制、十六进制或二进制表示。例如，以下指令都将十进制值109存入s5：
addi s5,x0,0b11011101 addi s5,x0,0x6D addi s5,x0,109

示例6.1 将高级代码翻译为汇编语言

将以下高级代码翻译为RISC-V汇编语言。假设变量a–c保存在寄存器s0–s2中，f–j保存在s3–s7中。

```
// 高级代码 a = b - c; f = (g + h) - (i + j);
```

解决方案 该程序使用了四条汇编语言指令。

```
# RISC-V 汇编代码
# s0 = a, s1 = b, s2 = c, s3 = f, s4 = g, s5 = h, s6 = i, s7 = j  sub s0, s1, s2 # a = b - c  add t0, s4, s5 # t0 = g + h  add t1, s6, s7 # t1 = i + j  sub s3, t0, t1 # f = (g + h) - (i + j)
```

寄存器组

表6.1列出了32个RISC-V寄存器的名称和用途。寄存器编号为0至31，并赋予特殊名称以指示其常规用途。汇编指令通常使用特殊名称（例如s1）以提高清晰度，但也可使用寄存器编号（例如x9表示9号寄存器）。零寄存器始终保存常量0，写入其中的值会被丢弃。寄存器s0至s11（寄存器8–9和18–27）以及t0至t6（寄存器5–7和28–31）用于存储变量；ra和a0至a7在函数调用期间具有特殊用途，详见第6.3.7节。寄存器2至4也被称为s、gp和tp，将在后文中描述。

常量/立即数

除了寄存器操作外，RISC-V指令还可以使用常量或immediate操作数。这些常量被称为immediates，因为它们的值可以直接从指令中获取，无需访问寄存器或内存。代码示例6.6展示了加法立即数指令addi，它将一个立即数与寄存器相加。在汇编代码中，立即数可以用十进制、十六进制或二进制表示。RISC-V汇编语言中的十六进制常量以0x开头，二进制常量以0b开头，与C语言相同。立即数是12位的二进制补码数，因此会被符号扩展为32位。addi指令是用小常量初始化寄存器值的一种有效方式。代码示例6.7分别将变量i、x和y初始化为0、2032和-78。

表6.1 RISC-V寄存器集

Name	Register Number	Use
zero	x0	Constant value 0
ra	x1	Return address
sp	x2	Stack pointer
gp	x3	Global pointer
tp	x4	Thread pointer
t0-2	x5-7	Temporary registers
s0/fp	x8	Saved register/Frame pointer
s1	x9	Saved register
a0-1	x10-11	Function arguments/Return values
a2-7	x12-17	Function arguments
s2-11	x18-27	Saved registers
t3-6	x28-31	Temporary registers

代码示例6.6 立即操作数

High-Level Code	RISC-V Assembly Code
<pre>a = a + 4; b = a - 12;</pre>	<pre># s0 = a, s1 = b addi s0, s0, 4 # a = a + 4 addi s1, s0, -12 # b = a - 12</pre>

代码示例6.7 使用立即数初始化值

High-Level Code	RISC-V Assembly Code
<pre>i = 0; x = 2032; y = -78;</pre>	<pre># s4 = i, s5 = x, s6 = y addi s4, zero, 0 # i = 0 addi s5, zero, 2032 # x = 2032 addi s6, zero, -78 # y = -78</pre>

要创建更大的常量，请使用 *load upper immediate* 指令（lui）后跟一条立即数加法指令（addi），如代码示例 6.8 所示。lui 指令将一个 20 位立即数加载到指令的最高 20 位，并将最低有效位设置为零。

C语言中的int数据类型表示一个*signed*数，即二进制补码整数。C语言规范要求int的宽度至少为*at least*16位，但并未规定具体大小。大多数现代编译器（包括RV32I的编译器）使用32位，因此int表示的范围为 $[-2^{31}, 2^{31}-1]$ 。C语言还定义了*int32_t*作为32位二进制补码整数，但输入起来较为繁琐。

代码示例6.8 32位常量示例

High-Level Code	RISC-V Assembly Code
<pre>int a = 0xABCDE123;</pre>	<pre>lui s2, 0xABCDE # s2 = 0xABCDE000 addi s2, s2, 0x123 # s2 = 0xABCDE123</pre>

在创建大立即数时，如果addi中的12位立即数为负数（即位11为1），则lui中的高位立即数必须加1。请记住，addi *sign*会对12位立即数进行符号扩展，因此负数立即数的高20位将全部为1。由于全1在二进制补码中表示-1，将全1加到高位立即数上相当于从高位立即数中减去1。代码示例6.9展示了所需立即数为0xFEEDA987的情况。lui s2, 0xFEEDB将0xFEEDB000存入s2。所需的高20位立即数0xFEEDA被加1。0x987是-1657的12位表示，因此addi s2, s2, -1657将s2与符号扩展后的12位立即数（0xFEEDB000 + 0xFFFFF987 = 0xFEEDA987）相加，并将结果存入s2，符合预期。

代码示例 6.9 第1位为1的32位常量

1

High-Level Code	RISC-V Assembly Code
<pre>int a = 0xFEEDA987;</pre>	<pre>lui s2, 0xFEEDB # s2 = 0xFEEDB000 addi s2, s2, -1657 # s2 = 0xFEEDA987</pre>

记忆
如果寄存器是操作数的唯一存储空间，那么程序将局限于不超过32个变量的简单程序。然而，数据也可以存储在内存中。寄存器文件小而快，而内存更大且更慢。因此，频繁使用的变量保存在寄存器中。在RISC-V架构中，指令仅对寄存器进行操作，因此存储在内存中的数据必须先移动到寄存器中才能被处理。通过结合使用内存和寄存器，程序可以相当快速地访问大量数据。回顾第5.5节，内存被组织为数据数组。RV32I RISC-V架构使用32位内存地址和32位数据字。

RISC-V 使用 *byte-addressable* 内存。也就是说，内存中的每个字节都有一个唯一的地址，如图 6.1(a) 所示。一个 32 位字由四个 8 位字节组成，因此每个字地址都是 4 的倍数。

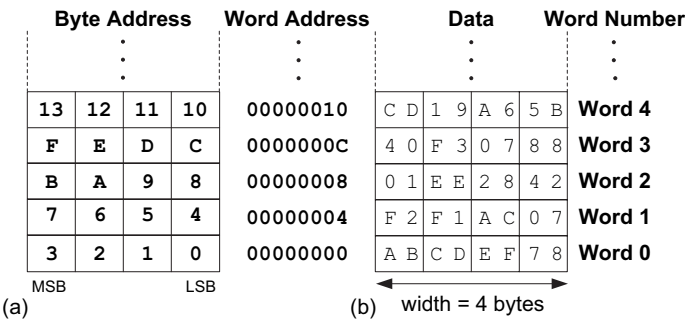


图6.1 RISC-V字节可寻址存储器展示：(a)字节地址和(b)数据

最高有效字节（MSB）位于左侧，最低有效字节（LSB）位于右侧。字内字节的顺序将在第6.6.1节进一步讨论。图6.1(b)中的32位字地址和数据值均以十六进制给出。例如，数据字0xF2F1AC07存储在内存地址4处。按照惯例，内存绘制时低地址位于底部，高地址位于顶部。

*load word*指令*lw*从内存中读取一个数据字到寄存器。代码示例6.10将位于地址8的内存字2加载到s7中。在C语言中，括号内的数字是*index*或字编号，我们将在6.3.6节进一步讨论。*lw*指令通过将*offset*与*base register*相加来指定内存地址。请记住，每个数据字为4字节，因此字地址是字编号的四倍。字编号0位于地址0，字1位于地址4，字2位于地址8，以此类推。在此示例中，基址寄存器*zero*与偏移量8相加，得到地址8，即字2。在代码示例6.10中执行加载字指令(*lw*)后，s7保存值0x01EE2842，这是图6.1中存储在内存地址8的数据值。

代码示例6.10 读取内存

High-Level Code	RISC-V Assembly Code
<pre>a = mem[2];</pre>	<pre># s7 = a lw s7, 8(zero) # s7 = data at memory address (zero + 8)</pre>

store word 指令`sw` 将寄存器中的数据字写入内存。代码示例 6.11 将寄存器 t3 中的值 42 写入位于地址 20 的内存字 5。

许多RISC-V实现要求 *word-aligned* ——即*lw*和*sw*的 *address*地址必须能被四整除。某些架构（如x86）允许非字对齐的数据读写，但其他架构为了简化设计而要求严格对齐。在本教材中，我们将假设采用严格对齐。当然，用于加载字节和存储字节的*lb*和*sb*指令（详见第6.3.6节）的字节地址无需字对齐。

代码示例6.11 写入内存

High-Level Code	RISC-V Assembly Code
<pre>mem[5] = 42;</pre>	<pre>addi t3, zero, 42 # t3 = 42 sw t3, 20(zero) # data value at memory address 20 = 42</pre>



凯瑟琳·约翰逊，1918–2020 克里奥拉·凯瑟琳·约翰逊是一位数学家兼计算机科学家，也是美国国家航空航天局（NASA）首批非裔女性员工之一。18岁时，她以最优等成绩从西弗吉尼亚大学毕业，获得数学和法语双学士学位。加入NASA后，她担任“计算机”一职——这是一个主要由女性组成的群体，负责进行精确计算。1961年，约翰逊计算了美国首位进入太空的宇航员艾伦·谢泼德的飞行轨迹。在NASA早期，即使女性完成了大部分工作，也不被鼓励在报告上署名。她的NASA同事信任她的计算能力，因此约翰逊通过验证电子计算机的计算结果，推动了电子计算机的采用。2015年，巴拉克·奥巴马总统授予她总统自由勋章。

6.3 编程

诸如C或Java之类的软件语言被称为*high-level programming languages*，因为它们是在比汇编语言更抽象的层次上编写的。许多高级语言使用常见的软件结构，例如算术和逻辑运算、if/else语句、for和while循环、数组索引以及函数调用。有关这些结构在C语言中的更多示例，请参见附录C。在本节中，我们首先讨论支持这些高级结构的程序流程和指令。然后，我们探讨如何将高级结构翻译成RISC-V汇编代码。

6.3.1 程序流程

与数据类似，指令也存储在内存中。每条指令长度为32位（4字节），我们将在第6.4节中讨论这一点，因此连续指令的地址每次增加4。例如，在下面的代码片段中，addi指令位于内存地址0x538处，而下一条指令lw位于地址0x53C处。

Memory address	Instruction
0x538	addi s1, s2, s3
0x53C	lw t2, 8(s1)
0x540	sw s3, 3(t6)

program counter（也称为PC）用于跟踪当前指令。PC保存当前指令的内存地址，并在每条指令执行完成后递增4，以便处理器能够从内存中读取或*fetch*下一条指令。例如，当addi指令正在执行时，PC为0x538。addi执行完毕后，PC递增4至0x53C，处理器随后获取该地址处的lw指令。

6.3.2 逻辑、移位和乘法指令

RISC-V架构定义了多种逻辑和算术指令。我们在此简要介绍这些指令，因为它们对于实现更高级别的结构是必要的。

逻辑指令

RISC-V *logical operations* 包含与、或和异或指令。这些指令分别对两个源寄存器进行按位运算，并将结果写入目标寄存器，如图6.2所示。这些逻辑运算的立即数版本——`andi`、`ori`和`xori`，则使用一个源寄存器和一个12位符号扩展立即数。¹

		源寄存器			
s1		0100 0110	1010 0001	1111 0001	1011 0111
s2		1111 1111	1111 1111	0000 0000	0000 0000
		结果			
and s3, s1, s2	s3	0100 0110	1010 0001	0000 0000	0000 0000
or s4, s1, s2	s4	1111 1111	1111 1111	1111 0001	1011 0111
xor s5, s1, s2	s5	1011 1001	0101 1110	1111 0001	1011 0111

图6.2 逻辑运算

`and`指令用于清零*clearing*或*masking*位（即强制将位设为0）。例如，图6.2中的`and`指令清除了s1中与s2低位对应的位。在此例中，s1的低两个字节被清零。s1未屏蔽的高两个字节0x46A1被存入s3。寄存器中任意子集的位都可以被清零。例如，要清除s0的第3位并将结果存入s6，可使用`andi s6, s0, 0xFF7`。

`or`指令用于合并两个寄存器中的位域。例如，0x347A0000 OR 0x000072FC = 0x347A72FC。它也可用于设置寄存器中的位（即强制某位为1）。例如，要将s0的第5位设置为1并将结果存入s7，可使用`ori s7, s0, 0x020`。

逻辑非操作可以通过 `xori s8, s1, -1` 实现。注意 -1 (0xFFFF) 会被符号扩展为 0xFFFFFFFF（全1）。与全1进行异或运算会翻转所有位，因此s8将得到s1的按位取反结果。`

移位指令

*Shift instructions*将寄存器中的值左移或右移，丢弃末尾的位。RISC-V的移位操作包括sll（逻辑左移）、srl（逻辑右移）和sra（算术右移）。如第5.2.5节所述，左移总是用零填充最低有效位。然而，右移可以是*logical*（零移入最高有效位）或*arithmetic*（符号位移入最高有效位）。这些移位操作在第二个源寄存器中指定移位量。每条指令也有立即数版本（slli、srli和srai），其中移位量由5位无符号立即数指定。

RISC-V基础指令集目前除移位操作外，不包含位操作指令。某些指令集架构还包含循环移位指令，以及其他位操作指令，如位清除、位设置等。截至2021年，RISC-V的“B”标准扩展（用于位操作）尚在规划中，尚未完成。

¹Sign-extended logical immediates are somewhat unusual. Many other architectures, such as MIPS and ARM, zero-extend the immediate for logical operations.

图6.3展示了当按立即数移位时，slli、srli和srai的汇编代码及结果寄存器值。s5按立即数移位，结果存入目标寄存器。

图6.3 具有立即移位量的移位指令

		Source register			
s5		1111 1111	0001 1100	0001 0000	1110 0111

Assembly code		Result			
slli t0, s5, 7	t0	1000 1110	0000 1000	0111 0011	1000 0000
srli s1, s5, 17	s1	0000 0000	0000 0000	0111 1111	1000 1110
srai t2, s5, 3	t2	1111 1111	1110 0011	1000 0010	0001 1100

如第5.2.5节所述，将数值左移 N 位相当于将其乘以 2^N 。例如，slli s0, s0, 3将s0乘以8（即 2^3 ）。同样，将数值右移 N 位相当于将其除以 2^N 。算术右移用于对二进制补码数进行除法，而逻辑右移用于对无符号数进行除法。

逻辑移位也与and和or指令一起使用，以提取或组装位字段。例如，以下代码从s7中提取第15:8位，并将其放入s6的低字节中。如果s7为0x1234ABCD，则此代码执行完毕后，s6将为0xAB。

```
srli s6, s7, 8 andi s6, s6,
0xFF
```

乘法指令*

乘法与其他算术运算略有不同，因为两个 N 位数相乘会产生一个 $2N$ 位的乘积。RISC-V架构提供了多种*multiply instructions*，可生成32位或64位的乘积。这些指令不属于RV32I，但包含在RVM（RISC-V乘/除）扩展中。

multiply 指令（mul）将两个32位数相乘，生成一个32位乘积。mul s1, s2, s3 将 s2 和 s3 中的值相乘，并将乘积的低32位存入 s1；乘积的高32位被丢弃。该指令适用于乘积能容纳在32位内的小数相乘。无论操作数被视为有符号还是无符号，乘积的低32位都是相同的。

“乘高”操作有三种版本：mulh、mulhsu 和 mulhu。这些指令将乘法结果的高 32 位存入目标寄存器。mulh (*multiply high signed signed*)

将两个操作数都视为有符号数。`mulhsu` (*multiply high signed unsigned*) 将第一个操作数视为有符号数, 第二个视为无符号数, 而`mulhu` (*multiply high unsigned unsigned*) 将两个操作数都视为无符号数。例如, `mulhsu t1, t2, t3`将`t2`视为32位有符号数(二进制补码), `t3`视为32位无符号数, 将这两个源操作数相乘, 并将结果的高32位存入`t1`。使用两条指令序列——一条“高位乘法”指令后跟`mul`指令——可以将32位乘法的完整64位结果存入用户指定的两个寄存器中。例如, 以下代码将`s3`和`s5`中的32位有符号数相乘, 并将64位乘积存入`t1`和`t2`。即, $\{t1, t2\} = s3 \times s5$ 。

```
mulh t1, s3, s5 mul t2,
s3, s5
```

6.3.3 分支

如果程序每次只能以相同的顺序运行, 且与输入无关, 那将会很乏味。计算机相对于计算器的一个优势在于其做出决策的能力。计算机根据输入执行不同的任务。例如, *if/else*语句、*switch/case*语句、*while*循环和*for*循环都会根据某些测试条件性地执行代码。*Branch*指令修改程序的流程, 使处理器能够获取内存中非顺序排列的指令。它们修改PC以跳过代码段或重复之前的代码。*Conditional branch*指令执行测试, 并且仅在测试结果为TRUE时进行分支。无条件分支指令, 称为*jumps*, 始终进行分支。

条件分支

RISC-V指令集有六条条件分支指令, 每条指令都使用两个源寄存器和一个指示跳转目标的标签。`beq` (*branch if equal*) 在两个源寄存器中的值相等时跳转。`bne` (*branch if not equal*) 在两个值不相等时跳转。`blt` (*branch if less than*) 在第一个源寄存器中的值小于第二个源寄存器中的值时跳转, 而`bge` (*branch if greater than or equal to*) 在第一个值大于或等于第二个值时跳转。`blt`和`bge`将操作数视为有符号数, 而`bltu`和`bgeu`则将操作数视为无符号数。

代码示例6.12展示了`beq`的使用。当程序执行到相等分支指令(`beq`)时, `s0`中的值等于`s1`中的值, 因此分支被采用。于是, 下一条执行的指令是紧跟在名为`target`的标签之后的`add`指令。分支与标签之间的`addi`和`sub`指令不会被执行。

无需`bgt`或`ble`, 因为这些可以通过交换`blt`和`bge`的源寄存器顺序得到。不过, 它们可作为伪指令使用(参见第6.3.8节)。

代码示例6.12 使用beq的条件分支

RISC-V Assembly Code

```
addi s0, zero, 4      # s0 = 0 + 4 = 4
addi s1, zero, 1      # s1 = 0 + 1 = 1
slli s1, s1, 2         # s1 = 1 << 2 = 4
beq  s0, s1, target    # s0 == s1, so branch is taken
addi s1, s1, 1         # not executed
sub  s1, s1, s0        # not executed
target:
add  s1, s1, s0        # s1 = 4 + 4 = 8
```

汇编代码使用*labels*来指示程序中的指令位置。标签指向其后的第一条指令。当汇编代码被翻译成机器码时，这些标签对应指令地址（将在第6.4.3节和第6.4.4节中讨论）。RISC-V汇编标签后跟冒号（:）。大多数程序员会对指令进行缩进，但不对标签缩进，以使标签更加醒目。

在代码示例6.13中，由于s0等于s1，分支未被执行，代码在bne（不相等时分支）指令后直接继续执行。该代码片段中的所有指令均被执行。

代码示例6.13 使用bne的条件分支

RISC-V Assembly Code

```
addi s0, zero, 4      # s0 = 0 + 4 = 4
addi s1, zero, 1      # s1 = 0 + 1 = 1
slli s1, s1, 2         # s1 = 1 << 2 = 4
bne  s0, s1, target    # branch not taken
addi s1, s1, 1         # s1 = 4 + 1 = 5
sub  s1, s1, s0        # s1 = 5 - 4 = 1
target:
add  s1, s1, s0        # s1 = 1 + 4 = 5
```

跳跃

程序可以跳转——即无条件分支——使用以下三条指令之一：*jump*（j）、*jump and link*（jal）或*jump register*（jr）。j指令直接跳转到指定标签处的指令。代码示例 6.14展示了如何使用j（跳转）指令跳过三条指令，并在标签target后的add指令处继续执行。执行j指令后，该程序无条件地继续执行标签target处的add指令。跳转指令与标签之间的所有指令均被跳过。我们将在第6.3.7节中讨论jal和jr指令，它们用于函数调用。

代码示例6.14 使用j的无条件分支

RISC-V Assembly Code		
j	target	# jump to target
srai	s1, s1, 2	# not executed
addi	s1, s1, 1	# not executed
sub	s1, s1, s0	# not executed
target:		
add	s1, s1, s0	# s1 = s1 + s0

6.3.4 条件语句

if、if/else 和 switch/case 语句是高级语言中常用的条件语句。它们各自有条件地执行由一条或多条语句组成的 *block* 代码块。本节展示如何将这些高级结构转换为 RISC-V 汇编语言。

条件语句

一个 *if* 语句仅在满足条件时执行一段代码块，即 *if block*。代码示例6.15展示了如何将 *if* 语句转换为 RISC-V 汇编代码。if 语句的汇编代码测试的是高级代码中条件的相反条件。在代码示例6.15中，高级代码测试的是 `apples == oranges`。汇编代码使用 `bne` 测试 `apples != oranges`，如果条件不满足则跳过 *if* 块。否则（即当 `apples == oranges` 时），分支不会执行，*if* 块将被执行。

在C语言及许多其他高级编程语言中，双等号==表示相等比较，若两边相等则返回TRUE。!=表示不等比较。

代码示例6.15 IF语句

High-Level Code	RISC-V Assembly Code
if (apples == oranges) f = g + h; apples = oranges - h;	# s0 = apples, s1 = oranges # s2 = f, s3 = g, s4 = h bne s0, s1, L1 # skip if (apples != oranges) add s2, s3, s4 # f = g + h L1: sub s0, s1, s4 # apples = oranges - h

If/else 语句

If/else 语句根据条件执行两个代码块之一。当 *if* 语句中的条件满足时，执行 *if* 块。否则，执行 *else* 块。代码示例6.16展示了一个 *if/else* 语句的示例。

与 *if* 语句类似，*if/else* 汇编代码测试的是高级代码中条件的相反条件。在代码示例6.16中，高级代码测试的是 `(apples == oranges)`，而汇编代码测试的是

代码示例6.16 IF/ELSE语句

High-Level Code	RISC-V Assembly Code
<pre>if (apples == oranges) f = g + h; else apples = oranges - h;</pre>	<pre># s0 = apples, s1 = oranges # s2 = f, s3 = g, s4 = h bne s0, s1, L1 # skip if (apples != oranges) add s2, s3, s4 # f = g + h j L2 L1: sub s0, s1, s4 # apples = oranges - h L2:</pre>

(苹果! =橙子)。如果相反条件为真，bne会跳过if块并执行else块。否则，if块执行并以跳转（j）越过else块结束。

Switch/case 语句*
Switch/case语句，也简称为case语句，根据条件执行多个代码块中的一个。如果没有条件满足，则执行default块。case语句等同于一系列嵌套的if/else语句。代码示例6.17

展示了两个功能相同的高级代码片段：它们计算是否从ATM（自动取款机）中吐出20美元、50美元或100美元。

代码示例 6.17 SWITCH/CASE 语句

High-Level Code	RISC-V Assembly Code
<pre>switch (button) { case 1: amt = 20; break; case 2: amt = 50; break; case 3: amt = 100; break; default: amt = 0; } // equivalent function using // if/else statements if (button == 1) amt = 20; else if (button == 2) amt = 50; else if (button == 3) amt = 100; else amt = 0;</pre>	<pre># s0 = button, s1 = amt case1: addi t0, zero, 1 # t0 = 1 bne s0, t0, case2 # button == 1? addi s1, zero, 20 # if yes, amt = 20 j done # break out of case case2: addi t0, zero, 2 # t0 = 2 bne s0, t0, case3 # button == 2? addi s1, zero, 50 # if yes, amt = 50 j done # break out of case case3: addi t0, zero, 3 # t0 = 3 bne s0, t0, default # button == 3? addi s1, zero, 100 # if yes, amt = 100 j done # break out of case default: add s1, zero, zero # amt=0 done:</pre>

根据按下的按钮（取款机）。两种高级代码片段的RISC-V汇编实现相同。

6.3.5 进入循环

循环根据条件重复执行一段代码。While循环和for循环在高级语言中常用。本节展示如何利用条件分支将它们翻译为RISC-V汇编语言。

While Loops
While循环会在满足条件时重复执行一段代码——即until条件not满足。代码示例6.18中的while循环确定了使得 $2^x=128$ 的x值。它执行了七次，直到`pow = 128`。

代码示例 6.18 WHILE 循环

High-Level Code	RISC-V Assembly Code
<pre>// determines the power // of x such that 2^x=128 int pow = 1; int x = 0; while (pow != 128) { pow = pow * 2; x = x + 1; }</pre>	<pre># s0 = pow, s1 = x addi s0, zero, 1 # pow = 1 add s1, zero, zero # x = 0 addi t0, zero, 128 # t0 = 128 while: beq s0, t0, done # pow = 128? slli s0, s0, 1 # pow = pow * 2 addi s1, s1, 1 # x = x + 1 j while # repeat loop done:</pre>

与 if/else 语句类似，while 循环的汇编代码测试的是高级代码中条件的相反情况。如果该相反条件为真（在此例中为 `s0 == 128`），则 while 循环结束。否则，不执行分支，循环体继续执行。代码示例 6.18 在 while 循环前将 `pow` 初始化为 1，`x` 初始化为 0。while 循环将 `pow` 与 128 比较，若相等则退出循环。否则，将 `pow` 加倍（使用左移），递增 `x`，并分支跳回 while 循环的开头。

*Do/while*do-while循环与while循环类似，但它们在检查条件之前会先执行一次循环体。代码示例6.19图示了这样一个循环。请注意，与之前的示例不同，该分支检查的条件与高级代码中的条件相同。

For Loops
在while循环之前初始化一个变量，在循环条件中检查该变量，并在每次循环中改变该变量，这种做法非常常见。

代码示例6.19 DO/WHILE循环

High-Level Code	RISC-V Assembly Code
<pre>// determines the power // of x such that 2^x = 128 int pow = 1; int x = 0; do { pow = pow * 2; x = x + 1; } while (pow != 128);</pre>	<pre># s0 = pow, s1 = x addi s0, zero, 1 # pow = 1 add s1, zero, zero # x = 0 addi t0, zero, 128 # t0 = 128 while: slli s0, s0, 1 # pow = pow * 2 addi s1, s1, 1 # x = x + 1 bne s0, t0, while # pow = 128? done:</pre>

通过while循环。*For*循环是一种便捷的简写形式，将初始化、条件检查和变量变化集中在一处。for循环的高级代码格式为：

for (初始化; 条件; 循环操作) 语句

初始化代码执行*before*，for循环开始。在*beginning of each*循环迭代时测试条件。如果条件不满足，循环退出。如果条件满足，则执行循环体中的语句。循环操作在每个循环迭代的*end*执行。

代码示例6.20将数字从0加到9。循环变量（本例中为*i*）初始化为0，并在每次循环迭代结束时递增。只要*i*小于10，for循环就会执行。请注意，此示例还展示了相对比较。循环检查<条件以继续执行，因此汇编代码检查相反条件>=以退出循环。

For循环对于访问存储在内存数组中的大量相似数据特别有用，接下来将讨论这一点。

代码示例6.20 FOR循环

High-Level Code	RISC-V Assembly Code
<pre>// add the numbers from 0 to 9 int sum = 0; int i; for (i = 0; i < 10; i = i + 1) { sum = sum + i; }</pre>	<pre># s0 = i, s1 = sum addi s1, zero, 0 # sum = 0 addi s0, zero, 0 # i = 0 addi t0, zero, 10 # t0 = 10 for: bge s0, t0, done # i >= 10? add s1, s1, s0 # sum = sum + i addi s0, s0, 1 # i = i + 1 j for # repeat loop done:</pre>

6.3.6 数组

为便于存储和访问，可将相似数据分组存入一个array中。数组将其内容存储在内存中的连续地址上。每个数组元素通过一个称为其index的数字来标识。数组中元素的数量称为数组的length。图 6.4展示了一个存储在内存中的200元素整数分数数组。每个连续元素的地址增加4，即一个整数所占的字节数。数组中第0th个元素的地址称为数组的base address。

代码示例6.21是一个分数膨胀算法，为每个分数增加10分。初始化分数数组的代码未显示。假设s0初始值为0x174300A0，即数组的基地址。数组的索引是一个变量(i)，每个数组元素递增1，因此我们在将其加到基地址之前先乘以4。

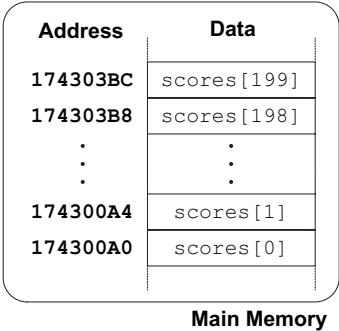


图6.4 内存中存放从基地址0x174300A0开始的scores[200]

代码示例6.21 使用FOR循环访问数组

High-Level Code	RISC-V Assembly Code
<pre>int i; int scores[200]; for (i = 0; i < 200; i = i + 1) scores[i] = scores[i] + 10;</pre>	<pre># s0 = scores base address, s1 = i addi s1, zero, 0 # i = 0 addi t2, zero, 200 # t2 = 200 for: bge s1, t2, done # if i >= 200 then done slli t0, s1, 2 # t0 = i * 4 add t0, t0, s0 # address of scores[i] lw t1, 0(t0) # t1 = scores[i] addi t1, t1, 10 # t1 = scores[i] + 10 sw t1, 0(t0) # scores[i] = t1 addi s1, s1, 1 # i = i + 1 j for # repeat done:</pre>

字节与字符
范围[-128, 127]内的数字可以存储在一个字节中，而不是整个字中。由于英语键盘的字符少于256个，英文字符通常使用字节来表示。C语言使用类型char来表示一个字节或字符。

早期计算机缺乏字节与英文字符之间的标准映射，因此计算机之间交换文本十分困难。1963年，美国标准协会发布了American Standard Code for Information Interchange (ASCII)，为每个文本字符分配了唯一的字节值。表6.2展示了这些字符

其他编程语言（如Java）使用不同的字符编码，最著名的是Unicode。Unicode使用16位来表示每个字符，因此它支持重音符号、变音符号以及亚洲语言。更多信息请参见www.unicode.org。

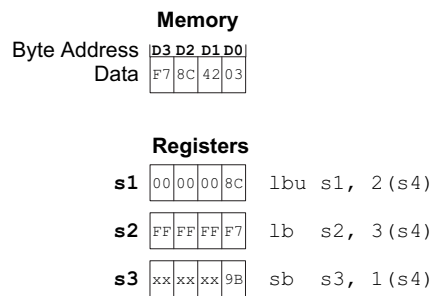
表6.2 ASCII编码

#	Char	#	Char	#	Char	#	Char	#	Char
20	space	30	0	40	@	50	P	60	`
21	!	31	1	41	A	51	Q	61	a
22	"	32	2	42	B	52	R	62	b
23	#	33	3	43	C	53	S	63	c
24	\$	34	4	44	D	54	T	64	d
25	%	35	5	45	E	55	U	65	e
26	&	36	6	46	F	56	V	66	f
27	'	37	7	47	G	57	W	67	g
28	(	38	8	48	H	58	X	68	h
29	)	39	9	49	I	59	Y	69	i
2A	*	3A	:	4A	J	5A	Z	6A	j
2B	+	3B	;	4B	K	5B	[	6B	k
2C	,	3C	<	4C	L	5C	\	6C	l
2D	-	3D	=	4D	M	5D	]	6D	m
2E	.	3E	>	4E	N	5E	^	6E	n
2F	/	3F	?	4F	O	5F	_	6F	o

可打印字符的编码。ASCII值以十六进制给出。小写字母和大写字母相差0x20（32）。

load byte (lb)、*load byte unsigned* (lbu) 和*store byte* (sb) 指令用于访问内存中的单个字节。lb对字节进行符号扩展，而lbu则将字节零扩展以填满整个32位寄存器。sb将32位寄存器的最低有效字节存储到内存中指定的字节地址。图 6.5展示了这三条指令，其中基地址s4为0xD0。lbu s1, 2(s4)将内存地址0xD2处的字节加载到s1的最低有效字节，并将寄存器剩余位填充为0。lb s2, 3(s4)将内存地址0xD3处的字节加载到s2的最低有效字节，并将该字节符号扩展到寄存器的上24位。sb s3, 1(s4)将s3的最低有效字节（0x9B）存储到内存字节地址0xD1，将0x42替换为0x9B。其他内存字节保持不变，s3的高位字节被忽略。

RISC-V还定义了lh、lhu和sh *half-word*加载和存储指令，这些指令对16位数据进行操作。这些指令的内存地址必须半字对齐。



一系列字符，例如单词或句子，称为`string`。字符串具有可变长度，因此编程语言必须提供一种确定字符串长度或结尾的方法。在C语言中，空字符（0x00）表示字符串的结束。例如，图6.6展示了字符串“Hello!”（0x48 65 6C 6C 6F 21 00）存储在内存中的情况。该字符串长度为七个字节，从地址0x1522FFF0延伸到0x1522FFF6。字符串的第一个字符（H = 0x48）存储在最低字节地址（0x1522FFF0）处。

图6.5 加载和存储字节的指令

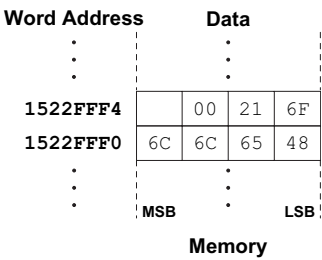


图6.6 字符串“Hello!”存储在内存中

示例6.2 使用lb和sb访问字符数组

以下高级代码通过将每个数组条目减去32，将10个字符的数组从小写转换为大写。将其翻译为RISC-V汇编语言。请注意，数组元素现在是1字节，而不是4字节，因此连续元素位于连续地址中。假设s0已保存chararray的基地址。

// 高级代码 // 之前已声明并初始化了 chararray[10] int i;

for (i = 0; i < 10; i = i + 1) chararray[i] = chararray[i] - 32;

解决方案

RISC-V 汇编代码 # s0 = 字符数组的基地址（先前已初始化），s1 = i addi s1, zero, 0 # i = 0 addi t3, zero, 10 # t3 = 10 for: bge s1, t3, done # i >= 10 ? add t4, s0, s1 # t4 = chararray[i] 的地址 lb t5, 0(t4) # t5 = chararray[i] addi t5, t5, -32 # t5 = chararray[i] - 32 sb t5, 0(t4) # chararray[i] = t5 addi s1, s1, 1 # i = i + 1 j for # 重复循环 done:

ASCII码源于早期的字符编码形式。自1838年起，电报机开始使用摩尔斯电码，通过一系列点（.）和划（-）来表示字符。例如，字母A、B、C、D分别表示为-、-...、-.-和-..。每个字母对应的点划数量各不相同。为了提高效率，常用字母采用较短的编码。

1874年，让-莫里斯-埃米尔·博多发明了一种5位编码，称为博多码。例如，A、B、C、D分别表示为00011、11001、01110和01001。然而，这种5位编码的32种可能编码不足以涵盖所有键盘字符，而8位编码则足够。因此，随着电子通信的普及，8位ASCII编码成为标准。

6.3.7 函数调用

高级语言支持*functions*（也称为*procedures*或*sub-routines*），以复用通用代码并使程序更加模块化和可读。函数可以有输入，称为*arguments*，以及输出，称为*return value*。函数应计算返回值，且不引起其他非预期的副作用。

当一个函数调用另一个函数时，调用函数（*caller*）和被调用函数（*callee*）必须就参数和返回值的存放位置达成一致。在RISC-V程序中，调用方通常在进行函数调用前将最多八个参数放入寄存器a0至a7中，而被调用方则在结束前将返回值放入寄存器a0中。遵循这一约定，即使调用方和被调用方由不同人员编写，双方也能知道参数和返回值的位置。

RISC-V实际上为返回值提供了两个寄存器，a0和a1。这允许64位的返回值，例如int64_t。

被调用者不得干扰调用者的行为。这意味着被调用者必须知道完成后应返回何处，且不得破坏调用者所需的任何寄存器或内存。调用者在通过跳转并链接指令（jal）跳转到被调用者的同时，将返回地址存储在*return address register* ra中。被调用者不得覆盖调用者所依赖的任何架构状态或内存。具体而言，被调用者必须保持*saved registers*（s0–s11）、返回地址（ra）以及*stack*（用于临时变量的部分内存）不变。

本节展示如何调用函数并从函数返回。同时说明函数如何访问参数和返回值，以及如何使用栈存储临时变量。

函数调用与返回

RISC-V 使用 *jump and link* 指令（jal）调用函数，并使用 *jump register* 指令（jr）从函数返回。代码示例 6.22 展示了 main 函数调用 simple 函数的过程。main 是调用者，simple 是被调用者。simple 函数

代码示例6.22 简单函数调用

High-Level Code	RISC-V Assembly Code
<pre>int main() { simple(); ... } // void means the function // returns no value void simple() { return; }</pre>	<pre>0x00000300 main: jal simple # call function 0x00000304 0x0000051c simple: jr ra # return</pre>

被调用时没有输入参数，也不产生返回值；它只是返回到调用者。在代码示例6.22中，每条RISC-V指令的左侧给出了示例指令地址。

jal和jr ra是函数调用和返回所需的两个基本指令。在代码示例6.22中，main函数通过执行jal simple来调用simple函数，该指令执行两项任务：跳转到位于simple（0x0000051C）的目标指令，并将return address（即jal之后指令的地址，此处为0x00000304）存储在返回地址寄存器（ra）中。程序员可以指定哪个寄存器写入返回地址，但默认是ra。因此，jal simple等同于jal ra, simple，并且是首选风格。simple函数通过执行jr ra指令立即返回，该指令跳转到ra中保存的指令地址。随后main函数在该地址（0x00000304）继续执行。

当前执行指令的指令地址保存在程序计数器PC中。因此，下一条指令地址被称为PC+4。

输入参数与返回值
代码示例6.22中的简单函数并不十分有用，因为它从调用函数（main）接收不到任何输入，也不返回任何输出。按照RISC-V的约定，函数使用a0到a7作为输入参数，并使用a0作为返回值。在代码示例6.23中，函数diffofsums被调用时带有四个参数，并返回一个结果。result是一个局部变量，我们选择将其保存在s3中。（寄存器的保存与恢复将在稍后讨论。）

根据RISC-V约定，调用函数main在调用函数之前，将函数参数从左到右放入输入寄存器a0至a7中。被调用函数diffofsums存储

j 和 jr 是 *pseudoinstructions*。它们不属于指令集的一部分，但方便程序员使用。RISC-V 汇编器会将它们替换为实际的 RISC-V 指令。汇编器将 j target 替换为 jal zero, target，该指令通过将返回地址写入零寄存器来跳转并丢弃返回地址；汇编器将 jr ra 替换为 jalr zero, ra, 0。

代码示例 6.23 带参数和{v*}的函数调用返回值

High-Level Code	RISC-V Assembly Code
<pre>int main(){ int y; ... y = diffofsums(2, 3, 4, 5); ... }</pre> <pre>int diffofsums(int f, int g, int h, int i){ int result; result = (f + g) - (h + i); return result; }</pre>	<pre># s7 = y main: ... addi a0, zero, 2 # argument 0 = 2 addi a1, zero, 3 # argument 1 = 3 addi a2, zero, 4 # argument 2 = 4 addi a3, zero, 5 # argument 3 = 5 jal diffofsums # call function add s7, a0, zero # y = returned value ... # s3 = result diffofsums: add t0, a0, a1 # t0 = f+g add t1, a2, a3 # t1 = h+i sub s3, t0, t1 # result = (f+g)-(h+i) add a0, s3, zero # put return value in a0 jr ra # return to caller</pre>

跳转并链接寄存器指令（jalr）与jal类似，但它从寄存器中获取目标地址，并可选择加上一个12位有符号立即数。例如，jalr ra, s1, 0x4C 跳转到地址 s1 + 0x4C，并将 PC+4 存入 ra。

栈通常以倒置方式存储在内存中，即栈顶实际位于最低地址，栈向下朝更低内存地址增长。这被称为*descending stack*。某些架构也支持向更高内存地址增长的*ascending stacks*。栈指针（sp）通常指向栈顶元素，这被称为*full stack*。某些架构（如ARM）还允许*empty stacks*，即sp指向栈顶上方一个字的地址。RISC-V架构定义了函数传递变量和使用栈的标准方式，以便不同编译器开发的库能够互操作。它规定了*full* 栈，本章将采用此方式。

返回值位于返回寄存器a0中。当调用一个参数超过八个的函数时，额外的输入参数会被放置在栈上，我们接下来将讨论这一点。

堆栈
栈是一种用作临时存储空间的内存——即在函数内部保存临时信息。当处理器需要更多临时空间时，栈会扩展（使用更多内存）；当处理器不再需要其中存储的变量时，栈会收缩（使用更少内存）。在解释函数如何利用栈存储临时值之前，我们先说明栈的工作原理。

栈是一种后进先出（LIFO）队列。就像一叠盘子，最后放入（或放置）到栈上的项*pushed*（最上面的盘子）是第一个可以被取出（移除）的项*popped*。每个函数可以分配栈空间来存储局部变量或用作临时空间，但函数必须在返回前释放该空间。*top of the stack*是最近分配的空间。与一叠盘子向上增长不同，RISC-V栈在内存中向下增长。也就是说，当程序需要更多临时空间时，栈会向更低的内存地址扩展。

图6.7展示了一张栈的图片。栈指针sp（寄存器2）是一个普通的RISC-V寄存器，按照惯例，它*points to top of the stack*。指针是内存地址的一种别称。sp指向（给出地址）数据。例如，在图6.7(a)中，栈指针sp保存地址值0xBEFFFAE8，并指向数据值0xAB000001。

堆栈指针（sp）从高内存地址开始，并根据需要递减以扩展。图6.7(b)展示了堆栈扩展以允许临时存储两个额外的数据字。为此，sp递减8，变为0xBEFFFAE0。两个额外的数据字0x12345678和0xFFEEDDCC被临时存储在堆栈上。

栈的重要用途之一是保存和恢复函数所使用的寄存器。请记住，函数应计算返回值，但不应产生其他非预期的副作用。特别地，一个

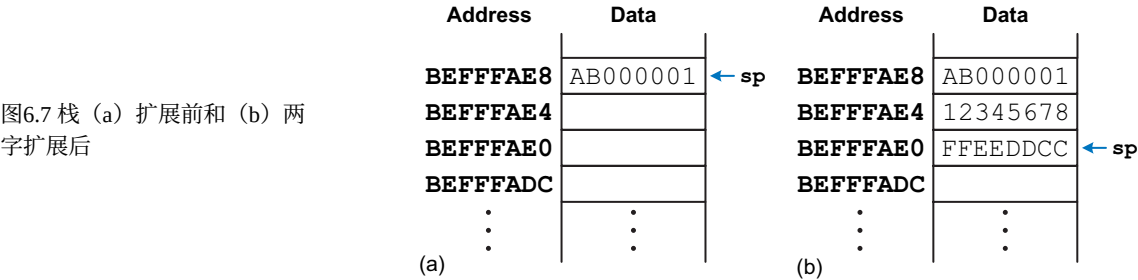


图6.7 栈 (a) 扩展前和 (b) 两字扩展后

函数不应修改除a0（包含返回值的寄存器）之外的任何寄存器。代码示例6.23中的diffofsums函数违反了此规则，因为它修改了t0、t1和s3。如果在调用diffofsums之前，main函数正在使用这些寄存器，那么这些寄存器的内容将被该函数调用破坏。

为解决此问题，函数在修改寄存器前将其保存到栈中，并在返回前从栈中恢复它们。具体步骤如下：

1. 在栈上腾出空间以存储一个或多个r的值

2. 将寄存器的值存储到栈上

3. 使用寄存器执行函数

4. 从栈上恢复寄存器的原始值

5. 释放栈上的空间
- 寄存器

代码示例6.24展示了diffofsums的改进版本，该版本保存并恢复t0、t1和s3。图6.8展示了栈在操作前、操作中的状态，

代码示例6.24 将寄存器保存到栈中的函数

High-Level Code	RISC-V Assembly Code
<pre>int diffofsums(int f, int g, int h, int i){ int result; result = (f + g) - (h + i); return result; }</pre>	<pre># s3 = result diffofsums: addi sp, sp, -12 # make space on stack to # store three registers sw s3, 8(sp) # save s3 on stack sw t0, 4(sp) # save t0 on stack sw t1, 0(sp) # save t1 on stack add t0, a0, a1 # t0 = f + g add t1, a2, a3 # t1 = h + i sub s3, t0, t1 # result = (f + g) - (h + i) add a0, s3, zero # put return value in a0 lw s3, 8(sp) # restore s3 from stack lw t0, 4(sp) # restore t0 from stack lw t1, 0(sp) # restore t1 from stack addi sp, sp, 12 # deallocate stack space jr ra # return to caller</pre>

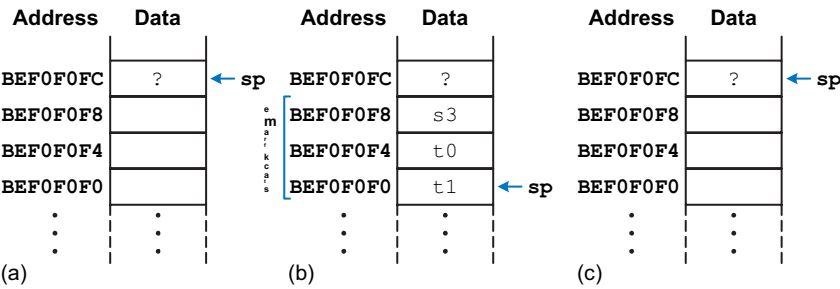


图6.8 栈：在diffofsums函数调用 (a) 之前、 (b) 期间和 (c) 之后

将寄存器值保存堆栈上称为 *pushing* 寄存器入栈。从堆栈中恢复寄存器值称为 *popping* 寄存器出栈。

在调用代码示例6.24中的diffofsums函数之后。栈起始地址为0xBEF0F0FC。diffofsums通过将栈指针sp减12来在栈上为三个字分配空间。然后将t0、t1和s3中当前的值存储在新分配的空间中。接着执行函数的其余部分，改变这三个寄存器中的值。在函数末尾，diffofsums从栈中恢复这些寄存器的值，释放其栈空间，并返回。当函数返回时，a0保存结果，但没有任何其他副作用：t0、t1、s3和sp的值与函数调用前相同。

函数为自己分配的栈空间称为其 *stack frame*。diffofsums的栈帧深度为三个字。模块化原则告诉我们，每个函数应仅访问自己的栈帧，而不应访问属于其他函数的栈帧。

保留寄存器
代码示例6.24假设所有使用的寄存器（t0、t1和s3）都必须保存和恢复。如果调用函数不使用这些寄存器，那么保存和恢复它们的努力就白费了。为了避免这种浪费，RISC-V将寄存器分为*preserved*和*nonpreserved*两类。保留寄存器在调用函数的开始和结束时必须包含相同的值，因为调用者期望保留寄存器的值在调用后保持不变。

保留的寄存器是s0到s11（因此得名*saved*）、sp和ra。非保留的寄存器，也称为*scratch*寄存器，是t0到t6（因此得名*temporary*）以及a0到a7，即参数寄存器。函数可以自由更改非保留的寄存器，但必须保存和恢复其使用的任何保留寄存器。

代码示例6.25展示了diffofsums的进一步改进版本，该版本仅在栈上保存s3。t0和t1是非保留寄存器，因此无需保存。

代码示例6.25 保存保留寄存器的函数关于栈

```
RISC-V Assembly Code

# s3 = result
diffofsums:
    addi sp, sp, -4      # make space on stack to store one register
    sw   s3, 0(sp)       # save s3 on stack
    add  t0, a0, a1      # t0 = f + g
    add  t1, a2, a3      # t1 = h + i
    sub  s3, t0, t1      # result = (f + g) - (h + i)
    add  a0, s3, zero     # put return value in a0
    lw   s3, 0(sp)       # restore s3 from stack
    addi sp, sp, 4       # deallocate stack space
    jr   ra              # return to caller
```

表6.3 保留与非保留的寄存器和内存

Preserved (<i>callee</i> -saved)	Nonpreserved (<i>caller</i> -saved)
Saved registers: s0-s11	Temporary registers: t0-t6
Return address: ra	Argument registers: a0-a7
Stack pointer: sp	
Stack above the stack pointer	Stack below the stack pointer

由于被调用函数可以自由更改任何非保留寄存器，调用方必须在进行函数调用前保存包含关键信息的非保留寄存器，并在调用后恢复这些寄存器。基于这些原因，保留寄存器也被称为*callee-saved*，而非保留寄存器则被称为*caller-saved*。

表6.3总结了哪些寄存器是被保留的。寄存器是否被保留的约定是RISC-V架构标准调用约定²的一部分，而非架构本身的一部分。

s0到s11通常用于保存函数内的局部变量，因此必须保存。ra也必须保存，以便被调用者知道返回位置。t0到t6用于保存临时结果。这些计算通常在函数调用之前完成，因此它们不会在函数调用过程中保留，调用者也很少需要保存它们。a0到a7在调用函数的过程中经常被覆盖。因此，如果调用者在被调用函数返回后依赖于自己的任何参数，则必须由调用者保存它们。

栈指针上方的栈空间会自动保留，只要被调用函数不向sp以上的内存地址写入数据。这样，它就不会修改任何其他函数的栈帧。栈指针本身也会被保留，因为被调用函数在返回前会通过加回与函数开始时从sp减去的相同数量来释放其栈帧。

敏锐的读者或优化编译器可能会注意到，diffofsums的局部变量result在返回后未用于其他任何用途。因此，我们可以消除该变量，直接将计算结果存入返回寄存器a0，从而无需在栈帧上分配空间，也无需将结果从s3移至a0。代码示例6.26展示了进一步优化后的diffofsums函数。

²From the RISC-V Instruction Set Manual, Volume I, version 2.2 © 2017.

代码示例6.26 优化后的diffofsums函数

RISC-V 汇编代码

```
# a0 = 结果 diffofsums: add t0, a0, a1 # t0 = f + g add t1,
a2, a3 # t1 = h + i sub a0, t0, t1 # 结果 = (f + g) - (h + i)
jr ra # 返回调用者
```

非叶函数调用

不调用其他函数的函数称为*leaf function*；diffofsums就是一个例子。调用其他函数的函数称为*nonleaf function*。非叶函数稍微复杂一些，因为它们可能需要在调用其他函数之前将非保留寄存器保存到账上，然后在调用之后恢复这些寄存器。具体来说，它们必须遵循以下规则：

调用者保存规则：在函数调用前，调用者必须保存调用后仍需使用的任何非保留寄存器（t0–t6 和 a0–a7）。调用后，必须在使用这些寄存器之前恢复它们。

被调用者保存规则：在被调用者修改任何保留寄存器（s0–s11 和 ra）之前，必须保存这些寄存器。在返回之前，必须恢复这些寄存器。

代码示例6.27展示了一个非叶子函数f1和一个叶子函数f2，包括所有必要的寄存器保存与恢复操作。f1将i保存在s4中，将x保存在s5中；f2将r保存在s4中。f1使用了被调用者保存的寄存器s4、s5和ra，因此根据被调用者保存规则，它首先将它们压入栈中。它使用t3来保存中间结果(a–b)，这样就不需要为此计算额外保存另一个寄存器。在调用f2之前，f1根据调用者保存规则将a0和a1保存到栈中，因为这些是非保留寄存器，f2可能会修改它们，而f1在调用后仍需要它们。ra会发生变化，因为调用f2会覆盖它。尽管t3也是非保留寄存器，f2可能覆盖它，但f1不再需要t3，因此不必保存它。然后f1将参数传递给a0中的f2，进行函数调用，并在a0中获取结果。f1随后恢复a0和a1，因为它仍然需要它们。当f1执行完毕时，它将返回值放入a0，恢复寄存器s4、s5、ra和sp，然后返回。f2根据被调用者保存规则保存并恢复s4（以及sp）。

A nonleaf function overwrites ra when it calls another function using jal. Thus, a nonleaf function must always save ra on its stack and restore it before returning.

仔细检查后，可能会注意到f2并未修改a1，因此f1无需保存和恢复它。然而，编译器并不总能轻易确定在函数调用期间哪些非保留寄存器可能会被干扰。因此，简单的编译器总是会让调用者保存和恢复其在调用后所需的任何非保留寄存器。

代码示例6.27 非叶函数调用

High-Level Code	RISC-V Assembly Code
<pre>int f1(int a, int b) { int i, x; x = (a + b)*(a - b); for (i = 0; i < a; i++) x = x + f2(b + i); return x; }</pre>	<pre># a0 = a, a1 = b, s4 = i, s5 = x f1: addi sp, sp, -12 # make room on stack for 3 registers sw ra, 8(sp) # save preserved registers used by f1 sw s4, -4(sp) sw s5, 0(sp) add s5, a0, a1 # x = (a + b) sub t3, a0, a1 # temp = (a - b) mul s5, s5, t3 # x = x * temp = (a + b) * (a - b) addi s4, zero, 0 # i = 0 for: bge s4, a0, return # if i >= a, exit loop addi sp, sp, -8 # make room on stack for 2 registers sw a0, 4(sp) # save nonpreserved regs. on stack sw a1, 0(sp) add a0, a1, s4 # argument is b + i jal f2 # call f2(b + i) add s5, s5, a0 # x = x + f2(b + i) lw a0, 4(sp) # restore nonpreserved registers lw a1, 0(sp) addi sp, sp, 8 addi s4, s4, 1 # i++ j for # continue for loop return: add a0, zero, s5 # return value is x lw ra, 8(sp) # restore preserved registers lw s4, 4(sp) lw s5, 0(sp) addi sp, sp, 12 # restore sp jr ra # return from f1 # a0 = p, s4 = r f2: addi sp, sp, -4 # save preserved regs. used by f2 sw s4, 0(sp) addi s4, a0, 5 # r = p + 5 add a0, s4, a0 # return value is r + p lw s4, 0(sp) # restore preserved registers addi sp, sp, 4 # restore sp jr ra # return from f2</pre>

优化编译器可以观察到f2是一个叶子过程，并可以将r分配到一个非保留寄存器中，从而避免保存和恢复s4的需要。图6.9显示了函数执行期间的堆栈情况。在此示例中，堆栈指针最初从0xBEF7FF0C开始。

递归函数调用

recursive function 是一个调用自身的非叶子函数。递归函数既充当调用者又充当被调用者，必须同时保存保留寄存器和非保留寄存器。例如，阶乘函数可以写成递归函数。回顾 $factorial(n) = n \times (n - 1) \times (n - 2) \times \cdots \times 2 \times 1$ 。阶乘函数可以递归地写成 $factorial(n) = n \times factorial(n - 1)$ ，如代码示例 6.28 所示。



图6.9 堆栈：(a) 函数调用前，(b) f1执行期间，(c) f2执行期间

代码示例6.28 阶乘递归函数调用

High-Level Code	RISC-V Assembly Code	
<pre>int factorial(int n) { if (n <= 1) return 1; else return (n * factorial(n - 1)); }</pre>	<pre>0x8500 factorial: addi sp, sp, -8 # make room for a0, ra 0x8504 sw a0, 4(sp) 0x8508 sw ra, 0(sp) 0x850C addi t0, zero, 1 # temporary = 1 0x8510 bgt a0, t0, else # if n > 1, go to else 0x8514 addi a0, zero, 1 # otherwise, return 1 0x8518 addi sp, sp, 8 # restore sp 0x851C jr ra # return 0x8520 else: addi a0, a0, -1 # n = n - 1 0x8524 jal factorial # recursive call 0x8528 lw t1, 4(sp) # restore n into t1 0x852C lw ra, 0(sp) # restore ra 0x8530 addi sp, sp, 8 # restore sp 0x8534 mul a0, t1, a0 # a0 = n * factorial(n - 1) 0x8538 jr ra # return</pre>	

1的阶乘就是1本身。为了方便引用程序地址，我们假设程序从地址0x8500开始。根据被调用者保存规则，factorial是一个非叶函数，必须保存ra。根据调用者保存规则，factorial在调用自身后还需要n，因此必须保存a0。于是，它在开始时将这两个寄存器压入栈中。然后检查 $n \leq 1$ 。如果是，则将返回值1放入a0，恢复栈指针，并返回调用者。在这种情况下，它无需恢复ra，因为ra从未被修改。如果 $n > 1$ ，函数递归调用factorial(n-1)。接着从栈中恢复n的值和返回地址寄存器(ra)，执行乘法运算，并返回该结果。注意，该函数巧妙地将n恢复到t1中，以免覆盖返回值。乘法指令(mul a0, t1, a0)将n(t1)与返回值(a0)相乘，并将结果放入a0，即返回寄存器。

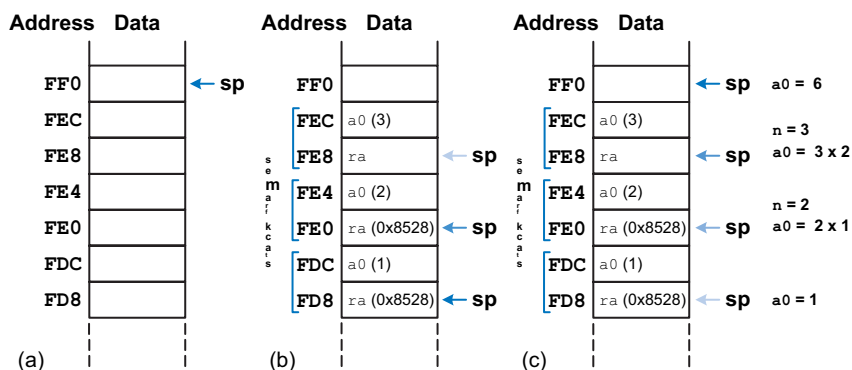


图6.10 栈: (a) 调用前, (b) 调用中, (c) 调用后, 阶乘函数调用时 $n = 3$

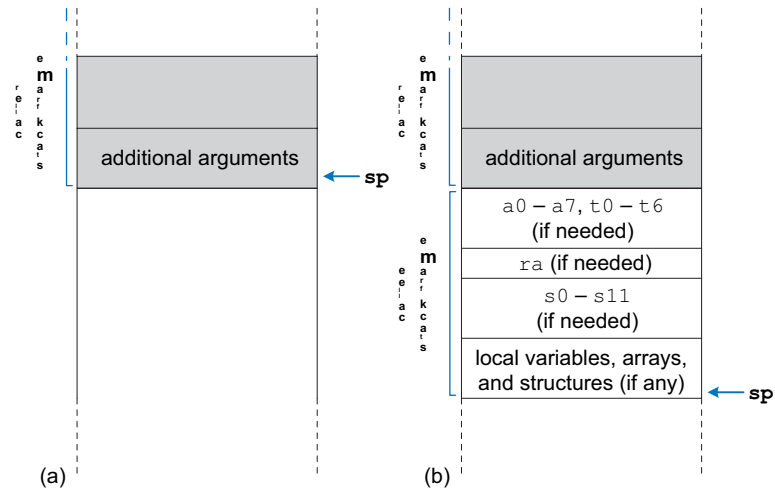
为清晰起见, 我们在函数调用开始时保存寄存器。优化编译器可能会注意到, 当 $n \leq 1$ 时, 无需保存 `a0` 和 `ra`, 因此仅在函数的 `else` 部分将寄存器保存到栈中。

图6.10展示了执行`factorial(3)`时的堆栈情况。为便于说明, 我们假设`sp`初始指向`0xFF0` (高位地址位为0), 如图6.10(a)所示。该函数创建一个双字堆栈帧来保存 n (`a0`) 和`ra`。在第一次调用时, `factorial`将`a0` (保存 $n = 3$) 保存在`0xFEC`, 将`ra`保存在`0xFE8`, 如图6.10(b)所示。随后函数将 n 改为2, 并递归调用`factorial(2)`, 使`ra`保存`0x8528`。在第二次调用时, 它将`a0` (保存 $n = 2$) 保存在`0xFE4`, 将`ra`保存在`0xFE0`。此时, 我们知道`ra`中保存的是`0x8528`。函数随后将 n 改为1, 并递归调用`factorial(1)`。在第三次调用时, 它将`a0` (保存 $n = 1$) 保存在`0xFDC`, 将`ra`保存在`0xFD8`。此时, `ra`再次保存`0x8528`。第三次调用`factorial`返回`a0`中的值1, 并在返回第二次调用前释放堆栈帧。第二次调用将 n (存入`t1`) 恢复为2, 将`ra`恢复为`0x8528` (它恰好已经具有该值), 释放堆栈帧, 并返回`a0 = 2 × 1 = 2`给第一次调用。第一次调用将 n (存入`t1`) 恢复为3, 恢复`ra` (调用者的返回地址), 释放堆栈帧, 并返回`a0 = 3 × 2 = 6`。图6.10(c)展示了递归调用的函数返回时的堆栈情况。当`factorial`返回到调用者时, 堆栈指针回到其原始位置 (`0xFF0`), 指针上方的堆栈内容均未改变, 所有被保存的寄存器保持其原始值。`a0`保存返回值6。

附加参数和局部变量*

函数可能拥有超过八个输入参数, 也可能有太多局部变量而无法保存在保留寄存器中。此时会使用栈来

图6.11 扩展的栈帧与附加参数
(a) 调用前, (b) 调用后



存储这些信息。根据RISC-V约定，如果函数有超过八个参数，前八个参数照常通过参数寄存器（a0 – a7）传递。额外的参数则通过栈传递，位置恰好在sp之上。调用方必须扩展其栈以为这些额外参数腾出空间。图6.11(a)展示了调用方为调用一个超过八个参数的函数时的栈结构。

函数也可以声明局部变量或数组。局部变量在函数内部声明，且只能在该函数内部访问。局部变量存储在s0到s11中；如果函数有太多局部变量，它们也可以存储在函数的栈帧中。局部数组和结构体也存储在栈上。

图6.11(b)展示了被调用者栈帧的组织结构。该栈帧用于保存临时寄存器、参数寄存器和返回地址寄存器（若因后续函数调用需要保存），以及函数将要修改的任何已保存寄存器。它还保存局部数组和任何多余的局部变量。如果被调用者拥有超过八个参数，则需在调用者的栈帧中查找这些参数。访问额外的输入参数是函数能够访问自身栈帧之外栈数据的唯一例外情况。

某些函数还包含一个指向活动栈帧（即正在执行的函数的栈帧）的*frame pointer*。按照惯例，该地址保存在fp寄存器（x8）中，该寄存器也是一个保留寄存器。

6.3.8 伪指令

在展示如何将汇编代码转换为机器码（即1和0）之前，让我们重新审视伪指令。请记住，RISC-V是一种精简指令集计算机（RISC），因此通过保持指令数量较少来最小化指令大小和硬件复杂度。然而，RISC-V定义了伪指令，这些伪指令实际上并非RISC-V指令集的一部分，但被程序员和编译器广泛使用。当转换为机器码时，伪指令会被

表6.4 伪指令

Pseudoinstruction	RISC-V Instructions	Description	Operation
j label	jal zero, label	jump	PC = label
jr ra	jalr zero, ra, 0	jump register	PC = ra
mv t5, s3	addi t5, s3, 0	move	t5 = t3
not s7, t2	xori s7, t2, -1	one's complement	s7 = ~t2
nop	addi zero, zero, 0	no operation	
li s8, 0x7EF	addi s8, zero, 0x7EF	load 12-bit immediate	s8 = 0x7EF
li s8, 0x56789DEF	lui s8, 0x5678A addi s8, s8, 0xDEF	load 32-bit immediate	s8 = 0x56789DEF
bgt s1, t3, L3	blt t3, s1, L3	branch if >	if (s1 > t3), PC = L3
bgez t2, L7	bge t2, zero, L7	branch if ≥ 0	if (t2 ≥ 0), PC = L7
call L1	jal L1	call nearby function	PC = L1, ra = PC + 4
call L5	auipc ra, imm _{31:12} jalr ra, ra, imm _{11:0}	call far away function	PC = L5, ra = PC + 4
ret	jalr zero, ra, 0	return from function	PC = ra

翻译成一条或多条RISC-V指令。例如，我们已经讨论过跳转（j）伪指令，它被转换为以x0作为目标地址的跳转并链接（jal）指令——即不写入返回地址。我们还指出，逻辑非可以通过将源操作数与全1进行异或来实现。

表6.4给出了伪指令及其实现所用的RISC-V指令示例。例如，move指令（mv）将一个寄存器的内容复制到另一个寄存器。加载立即数伪指令（li）通过lui和addi指令的组合加载一个32位常量。如果该常量可以用12位表示，则li会被翻译成一条addi指令。空操作伪指令（nop，读作“no o p”）不执行任何操作。执行时PC增加4，但不会改变其他寄存器或内存值。call伪指令用于进行过程调用。如果调用的是附近的函数，call会被翻译成一条jalr指令。然而，如果函数距离较远，call会被翻译成两条RISC-V指令：auipc和jalr。例如，auipc s1, 0xABCDE将0xABCDE000加到PC上，并将结果存入s1。因此，如果PC为0x02000000，则s1现在持有0xAD CDE000。jalr ra, s1, 0x730随后跳转到地址s1 + 0x730（0xADCDE730），并将PC+4存入ra。ret伪指令用于从函数返回。

Nops 常用于在程序中生成精确的延迟。

它转换为 ``jalr x0, ra, 0``。附录B中的表B.7列出了最常见的RV32I伪指令。附录B印在本教材的内封面上。

6.4 机器语言

汇编语言便于人类阅读。然而，数字电路只能理解1和0。因此，用汇编语言编写的程序会从助记符翻译成仅使用1和0的表示形式，称为 *machine language*。本节将介绍RISC-V机器语言以及在汇编语言和机器语言之间进行转换的繁琐过程。

RISC-V 使用 32 位指令。同样，规整性支持简洁性，而最规整的选择是将所有指令编码为可存储在内存中的字。尽管某些指令可能不需要全部 32 位编码，但可变长度指令会增加复杂性。简洁性也会鼓励采用单指令格式，但这过于局限。然而，这个问题让我们得以引入最后一条设计原则：

设计原则4：好的设计需要好的权衡。

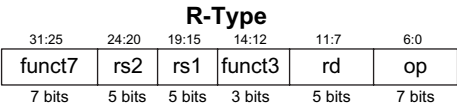
RISC-V 做出了折衷，定义了四种主要指令格式：*R-type*、*I-type*、*S/B-type* 和 *U/J-type*。这种少量的格式使得指令之间具有一定的规律性，从而简化解码器硬件，同时也适应了不同的指令需求。*R-type*（*register*）指令，例如 `add s0, s1, s2`，对三个寄存器进行操作。*I-type*（立即数）指令，例如 `addi s3, s4, 42`，以及 *S/B-type*（存储/分支）指令，例如 `sw a0, 4(sp)` 或 `beq a0,a1,L1`，对两个寄存器和一个12位或13位有符号立即数进行操作。*U/J-type*（高位立即数/跳转）指令，例如 `jal ra, factoria l`，对一个寄存器和一个20位或21位立即数进行操作。本节讨论这些 RISC-V 机器指令格式，并展示它们如何编码为二进制。附录 B 提供了所有 RV32I 指令的快速参考。

S/B型明显不被称为B/S型。

6.4.1 R型指令

R型（*register*型）指令使用三个寄存器作为操作数：两个作为源操作数，一个作为目标操作数。图6.12展示了R型机器

图6.12 R型指令格式



Assembly	Field Values						Machine Code					
	funct7	rs2	rs1	funct3	rd	op	funct7	rs2	rs1	funct3	rd	op
add s2, s3, s4	0	20	19	0	18	51	0000,000	10100	10011	000	10010	011,0011
add x18,x19,x20												(0x01498933)
sub t0, t1, t2	32	7	6	0	5	51	0100,000	00111	00110	000	00101	011,0011
sub x5, x6, x7												(0x407302B3)
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

图6.13 R型指令的机器码

指令格式。32位指令包含六个字段：funct7、rs2、rs1、funct3、rd和op。每个字段的长度为三到七位，如图所示。

指令执行的操作编码在三个以蓝色高亮的字段中：7位op（也称为操作码）和7位及3位的funct7与funct3（也称为功能字段）。具体的R型操作由操作码和功能字段共同决定。这些位合称为*control bits*，因为它们控制要执行的操作。例如，加法指令的操作码和功能字段为：op = 51 (0110 011₂)、funct7 = 0 (0000000₂) 和 funct3 = 0 (000₂)。类似地，减法指令的op = 51、funct7 = 32 (0100000₂) 和 funct3 = 0 (000₂)。图6.13展示了两个R型指令（add和sub）的机器码。两个源寄存器和目标寄存器编码在三个字段中：rs1、rs2和rd。这些字段包含表6.1中给出的寄存器编号。例如，s0是寄存器8（x8）。注意，在汇编语言和机器语言指令中，寄存器的顺序是相反的。例如，汇编指令add s2, s3, s4的rd = s2 (18)、rs1 = s3 (19)和rs2 = s4 (20)。这些寄存器在汇编指令中从左到右列出，但在机器指令中从右到左列出。

附录B中的表B.1列出了RV32I指令的操作码和功能字段（funct3和funct7）。从汇编语言翻译为机器码（如图6.13所示）最简单的方法是写出每个字段的值，并将这些值转换为二进制。然后，将位分组为每四位一组，转换为十六进制，使机器语言表示更加紧凑。

Appendix B is located on the inside covers of this book.

其他R型指令包括移位（sll、srl和sra）和逻辑运算（and、or和xor）。带有立即数移位量的移位指令（slli、srli和srai）属于I型指令，将在下一节6.4.2中讨论。

图6.14展示了左移逻辑（sll）和异或（xor）的机器码。所有R型操作的操作码均为51 (0110011₂)。对于使用寄存器移位量的移位指令（sll、srl和sra），将rs1移位rs2寄存器中第4:0位的无符号5位值，并将结果存入rd。所有移位指令中，funct7和funct3编码了要执行的移位或逻辑操作类型，如表B.1所示。对于sll，funct7 = 0且funct3 = 1；xor使用funct7 = 0和funct3 = 4。

Assembly		Field Values						Machine Code					
		funct7	rs2	rs1	funct3	rd	op	funct7	rs2	rs1	funct3	rd	op
sll	s7, t0, s1	0	9	5	1	23	51	0000 000	01001	00101	001	10111	011 0011
sll	x23,x5, x9												
xor	s8, s9, s10	0	26	25	4	24	51	0000 000	11010	11001	100	11000	011 0011
xor	x24,x25,x26												
		7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

R型指令有17位的操作码和功能码，足以表示 $2^{17} = 131,072$ 条不同的指令。考虑到目前我们定义的R型指令还不到12条，这似乎过于庞大了。然而，编码源寄存器和目标寄存器只需要另外15位。如此庞大的指令集空间使得RISC-V具有高度的可扩展性。例如，RISC-V 增加了浮点指令，详见第6.6.4节和附录B。

Extension

图6.14 R型指令的更多机器码

示例6.3 将R型汇编指令翻译为机器码

将以下RISC-V汇编指令翻译成机器语言：

将 t3 与 s4、s5 相加

解：根据表6.1，t3、s4和s5分别是寄存器28、20和21。根据表B.1，add的操作码为51（0110011₂），功能码funct7=为0，funct3=为0。因此，字段和机器码如图6.15所示。将机器语言写成十六进制的最简单方法是先将其写成二进制，然后查看每四位一组（对应十六进制数字，用蓝色下划线标出）。因此，该机器语言指令为0x015A0E33。

Assembly		Field Values						Machine Code					
		funct7	rs2	rs1	funct3	rd	op	funct7	rs2	rs1	funct3	rd	op
add	t3, s4, s5	0	21	20	0	28	51	0000,000	10101	10100	000	11100	011,0011
add	x28,x20,x21												
		7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

图6.15 示例6.3中R型指令的机器码

6.4.2 I型指令

I型（*immediate*）指令使用两个寄存器操作数和一个立即数操作数。I型指令包括addi、andi、ori、xori、加载指令（lw、lh、lb、lhu和lbu）以及寄存器跳转指令（jalr）。图6.16展示了I型机器指令格式。它与R型类似，但包含一个12位的立即数字段imm，取代了funct7和rs2字段。rs1和imm是源操作数，rd是目标寄存器。

图6.17展示了编码I型指令的几个示例。立即数字段表示一个12位有符号（二进制补码）

图6.16 I型指令格式



Assembly	Field Values					Machine Code					
	imm _{11:0}	rs1	funct3	rd	op	imm _{11:0}	rs1	funct3	rd	op	
addi s0, s1, 12	12	9	0	8	19	0000 0000 1100	01001	000	01000	001 0011	(0x00C48413)
addi x8, x9, 12											
addi s2, t1, -14	-14	6	0	18	19	1111 1111 0010	00110	000	10010	001 0011	(0xFF230913)
addi x18, x6, -14											
lw t2, -6(s3)	-6	19	2	7	3	1111 1111 1010	10011	010	00111	000 0011	(0xFFA9A383)
lw x7, -6(x19)											
lb s4, 0x1F(s4)	0x1F	20	0	20	3	0000 0001 1111	10100	000	10100	000 0011	(0x01FA0A03)
lb x20, 0x1F(x20)											
slli s2, s7, 5	5	23	1	18	19	0000 0000 0101	10111	001	10010	001 0011	(0x005B9913)
slli x18, x23, 5											
srai t1, t2, 29	(upper 7 bits =32) 29	7	5	6	19	0100 0001 1101	00111	101	00110	001 0011	(0x41D3D313)
srai x6, x7, 29											
	12 bits	5 bits	3 bits	5 bits	7 bits	12 bits	5 bits	3 bits	5 bits	7 bits	

图6.17 I型指令的机器码

对于所有I型指令，除了立即数移位指令（slli、srli和srai）之外，立即数均为有符号数。对于这些移位指令，imm_{4:0}是5位无符号移位量；对于srli和slli，立即数的高7位为0，但srai在imm₁₀（即指令位30）中置1，如图6.17所示。与R型指令一样，I型汇编指令中操作数的顺序与机器指令中的顺序不同。

示例6.4 将I型汇编指令翻译为机器码

将以下汇编指令翻译成机器语言。

```
lw t3, -36(s4)
```

根据表6.1，t3和s4分别是寄存器28和20。rs1（s4 = x20）指定基址，rd（t3 = x28）指定目标。立即数imm编码了12位偏移量（-36）。表B.1指出lw的操作码为3（0000011₂），funct3为2（010₂）。字段和机器码如图6.18所示。

I-type指令有一个12位的立即数字段，但这些立即数用于32位操作。例如，lw指令将一个12位偏移量加到32位基址寄存器上。那么32位中的高20位应该填入什么？对于正立即数，高位应全为0，但对于负立即数，

Assembly	Field Values					Machine Code					
	imm _{11:0}	rs1	funct3	rd	op	imm _{11:0}	rs1	funct3	rd	op	
lw t3, -36(s4)	-36	20	2	28	3	1111 1101 1100	10100	010	11100	000 0011	(0xFDCA2E03)
lw x28, -36(x20)											
	12 bits	5 bits	3 bits	5 bits	7 bits	12 bits	5 bits	3 bits	5 bits	7 bits	

图6.18 示例6.4中I型指令的机器码

请记住，一个M位的二进制补码数通过将M位数的符号位（最高有效位）复制到N位数的所有高位，从而符号扩展为N位数（ $N > M$ ）。对二进制补码数进行符号扩展不会改变其值。例如， 1101_2 是一个4位二进制补码数，表示 -3_{10} 。当符号扩展为8位时，它变为 11111101_2 ，仍然表示 -3_{10} 。

立即数，高位应全为1。回顾第1.4.6节，这被称为*sign extension*。

6.4.3 S/B型指令

与I型指令类似，S/B型（*store/branch*）指令使用两个寄存器操作数和一个立即数操作数。然而，在S/B型中，两个寄存器操作数均为源寄存器（rs1和rs2），而I型指令则使用一个源寄存器（rs1）和一个目标寄存器（rd）。图6.19展示了S/B型机器指令格式。该指令将R型指令中的funct7和rd字段替换为12位立即数imm。因此，该立即数字段被分割到指令的两个位范围中，即位31:25和位11:7。

存储指令使用S型，分支指令使用B型。S型和B型格式仅在立即数的编码方式上有所不同。S型指令编码一个12位有符号（二进制补码）立即数，其中高7位（imm_{11:5}）位于指令的31:25位，低5位（imm_{4:0}）位于指令的11:7位。

B型指令编码一个表示*branch offset*的13位有符号立即数，但指令中仅编码了其中的12位。最低有效位始终为0，因为分支偏移量总是偶数个字节（后续将解释）。B型指令的立即数采用一种略显奇特的位交错编码方式：imm₁₂位于instr₃₁；imm₁₁位于instr₇；imm_{10:5}位于instr_{30:25}；imm_{4:1}位于instr_{11:8}；而imm₀始终为0，因此不包含在指令中。这种位混合处理旨在尽可能使立即数位在不同指令格式中占据相同的指令位，同时确保符号位始终位于instr₃₁（详见第6.4.5节）。

图6.20展示了使用S型格式编码存储指令的几个示例。rs1是基地址，imm是偏移量，rs2是要存储到内存的值。请注意，负立即数使用12位二进制补码表示。例如，在sw x7, -6(x19)中，寄存器x19是基地址（rs1），x7是第二个源操作数（rs2），即要存储到内存的值，而-6是偏移量。对于所有S型指令，op为35（0100011₂），funct3用于区分sb（0）、sh（1）和sw（2）。

In the `sw` assembly instruction, `rs2` is the leftmost register, that is: `sw rs2, offset(rs1)`

图6.19 S型和B型指令格式

31:25	24:20	19:15	14:12	11:7	6:0	
imm _{11:5}	rs2	rs1	funct3	imm _{4:0}	op	S-Type
imm _{12,10:5}	rs2	rs1	funct3	imm _{4:1,11}	op	B-Type
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	

Assembly	Field Values						Machine Code					
	imm _{11:5}	rs2	rs1	funct3	imm _{4:0}	op	imm _{11:5}	rs2	rs1	funct3	imm _{4:0}	op
sw t2, -6(s3)	1111 111	7	19	2	11010	35	1111 111	00111	10011	010	11010	010 0011
sw x7, -6(x19)	0000 000	20	5	1	10111	35	0000 000	10100	00101	001	10111	010 0011
sh s4, 23(t0)	0000 001	30	0	0	01101	35	0000 001	11110	00000	000	01101	010 0011
sh x20,23(x5)												
sb t5, 0x2D(zero)												
sb x30,0x2D(x0)												
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

Figure 6.20 Machine code for S-type instructions

#Address

RISC-V Assembly

0x70

beq s0, t5, L1

0x74

add s1, s2, s3

0x78

sub s5, s6, s7

0x7C

lw t0, 0(s1)

0x80

L1: addi s1, s1, -15

1

2

3

4

L1 is 4 instructions (i.e., 16 bytes) past beq

imm_{12:0} = 16

0 0 0 0 0 0 0 0 1 0 0 00X

bit number

12 11 10 9 8 7 6 5 4 3 2 1 0

Assembly	Field Values						Machine Code					
	imm _{12:10:5}	rs2	rs1	funct3	imm _{4:1,11}	op	imm _{12:10:5}	rs2	rs1	funct3	imm _{4:1,11}	op
beq s0, t5, L1	0000 000	30	8	0	1000 0	99	0000 000	11110	01000	000	1000 0	110 0011
beq x8, x30, 16												
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

图6.21 B型指令格式及beq的计算

分支指令（beq、bne、blt、bge、bltu 和 bgeu）采用 B 型指令格式。图 6.21 展示了一些包含相等则分支指令 beq 的示例代码。每条指令左侧给出了指令地址。*branch target address* (BTA) 是分支的目标地址。图 6.21 中的 beq 指令的 BTA 为 0x80，即 L1 标签的指令地址。*branch offset* 经过符号扩展后，与分支指令的地址相加，得到分支目标地址。

对于B型指令，rs1和rs2是两个源寄存器，13位的立即数分支偏移量imm_{12:0}给出了分支指令与BTA之间的*number of bytes*。在这种情况下，BTA位于beq指令之后的四条指令处，即beq之后4×4=16字节的位置。因此，分支偏移量为16。由于分支偏移量的第0位始终为0，因此指令中仅编码了第12:1位。

对于32位指令，13位分支偏移量（imm_{12:0}）的第1:0位始终为零，因为32位指令占用4字节内存。因此，指令地址始终能被4整除，分支偏移量的第1位和第0位均无需编码到指令中。然而，RV32I仅省略了第0位。这实现了与16位（2字节）RISC-V压缩指令的兼容性（见第6.6.5节）。若处理器硬件同时支持两种指令长度，编译器便可混合使用16位和32位指令。

示例6.5 将B型汇编指令编码为机器码

考虑以下RISC-V汇编代码片段。指令地址写在每条指令的左侧。将条件不等跳转（bne）指令翻译为机器码。

Address	Instruction
0x354	L1: addi s1, s1, 1
0x358	sub t0, t1, s7
...	...
0xEB0	bne s8, s9, L1

解答 根据表6.1，s8和s9分别是寄存器24和25。因此，rs1为24，rs2为25。标签L1位于0xEB0 – 0x354 = 0xB5C（2908字节）*before* bne指令处。因此，13位立即数为–2908（1010010100100₂）。根据附录B，操作码为99（1100011₂），funct3为1（001₂）。因此，机器码如图6.22所示。注意，分支指令可以向前分支（到更高地址），或者如本例所示，向后分支（到更低地址）。

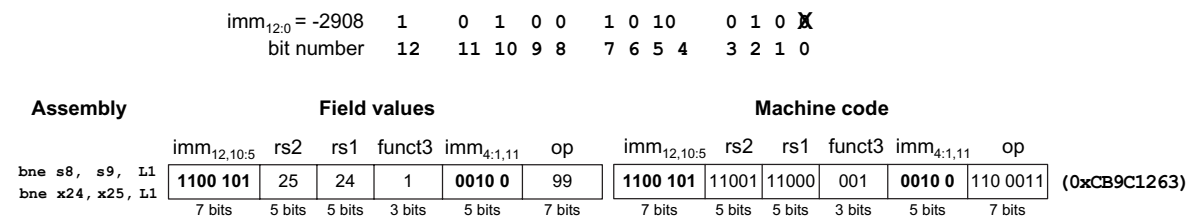


图6.22 示例6.5中B型指令的机器码

6.4.4 U/J型指令

U/J型（*upper immediate/jump*）指令有一个目标寄存器操作数rd和一个20位立即数字段，如图6.23所示。与其他格式一样，U/J型指令有一个7位操作码。在U型指令中，剩余位指定一个32位立即数的最高20位。在J型指令中，剩余20位指定一个21位立即跳转偏移量的最高20位。与B型指令一样，立即数的最低位始终为0，且不在J型指令中编码。

与B型指令类似，J型立即数字段的位序也以奇怪的方式打乱。计算机对此并不在意，但对人类而言却颇为恼人。

jalr 是 I 型 (not J 型!) 指令。
jal 是唯一的 J 型指令。

该指令因为始终为0，其余位被混洗到20位立即数字段中，如图6.25所示。如果jal汇编指令未指定目标寄存器rd，则该字段默认为ra (x1)。例如，指令jal L1等价于jal ra, L1，且rd = 1。普通跳转 (j) 编码为jal，且rd = 0。

6.4.5 立即数编码

RISC-V使用32位有符号立即数。指令中仅编码了立即数的12到21位。图6.26展示了每种指令类型如何形成立即数。I型和S型指令编码12位有符号立即数。J型和B型指令使用21位和13位有符号立即数，其中最低有效位始终为0（参见第6.4.3节和第6.4.4节）。U型指令编码32位立即数的高20位。

图 6.26 RISC-V 立即数

imm ₁₁												imm _{11:1}				imm ₀	I, S
imm ₁₂												imm _{11:1}				0	B
imm _{31:21}						imm _{20:12}						0					U
imm ₂₀						imm _{20:12}						imm _{11:1}				0	J

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0

在各种指令格式中，RV32I 试图将立即数位保持在相同的指令位上，以简化硬件设计（代价是使指令编码更复杂）。图 6.27 通过展示所有格式的指令字段突出了这种一致性（操作码是所有指令的第 6:0 位，未在图中显示）。instr₃₁ 始终保存立即数的符号位。对于 U 型指令，instr_{30:20} 保存 imm_{30:20}。否则，instr_{30:25} 保存 imm_{10:5}，instr_{19:12} 保存 imm_{19:12}（针对 U/J 型指令）。imm_{4:1} 占据 instr_{24:21} 或 instr_{11:8}。立即数的第 11 位（当它不是符号位时）和第 0 位是浮动位，位于指令的第 0 位或第 20 位。

图6.27 RISC-V机器指令中的立即数编码

11 10 9 8 7 6 5 4 3 2 1 0	rs1	funct3	rd	I
11 10 9 8 7 6 5	rs2	rs1	funct3 4 3 2 1 0	S
12 10 9 8 7 6 5	rs2	rs1	funct3 4 3 2 1 11	B
31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12	rd			U
20 10 9 8 7 6 5 4 3 2 1 11 19 18 17 16 15 14 13 12	rd			J

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7

保持指令格式中立即数字段位置的一致性，是规律性简化设计的另一个例子——具体而言，它最大限度地减少了提取和符号扩展立即数所需的导线和多路复用器数量。习题6.47和6.48将进一步探讨这一设计决策对硬件的影响。

6.4.6 寻址模式

addressing mode 定义了指令如何指定其操作数。本节概述了用于寻址指令操作数的模式。RISC-V使用四种主要模式：*register*、*immediate*、*base*和*PC-relative*寻址。大多数其他架构也提供类似的寻址模式，因此理解这些模式有助于您学习其他汇编语言。前三种模式（寄存器、立即数和基址寻址）定义了读写操作数的模式。最后一种模式（PC相对寻址）定义了写入程序计数器（PC）的模式。

仅寄存器寻址

Register-only addressing 所有源操作数和目标操作数均使用寄存器。所有R型指令仅采用寄存器寻址方式。

立即寻址

Immediate addressing 使用立即数以及寄存器作为操作数。某些I型指令，例如立即数加法（*addi*）和异或立即数（*xori*），采用12位有符号立即数的立即数寻址方式。具有立即数移位量的移位指令（*slli*、*srl*和*srai*）是I型指令，将5位无符号立即数移位量编码在 $\text{imm}_{4:0}$ 中。加载指令（*lb*、*lh*和*lw*）使用I型指令格式，但采用基址寻址，这将在下文讨论。

基址寻址

内存访问指令，如加载字（*lw*）和存储字（*sw*），使用*base addressing*。内存操作数的有效地址通过将寄存器 rs1 中的基地址与立即数字段中的符号扩展12位偏移量相加来计算。加载指令为I型指令，存储指令为S型指令。

PC相对寻址

分支、跳转与链接（*jal*）指令使用*PC-relative addressing*来指定PC的新值。立即数字段中编码的有符号偏移量被加到PC上，以得到目标地址，即新的PC；因此，目标地址被称为相对于当前PC。分支指令和*jal*分别使用13位和21位的有符号立即数。

跳转与链接寄存器（jalr）指令使用基址寻址，而非PC相对寻址。它能够跳转到32位地址空间中的任意指令地址，因为其目标地址由rs1与12位有符号立即数相加形成。返回地址PC+4被写入rd。

以下指令序列使该程序能够跳转到任意地址。每条指令的左侧列出了其指令地址。在此例中，程序跳转到地址0x12345678，并将0x0100FE7C（即PC+4）写入t1。

```
# 地址 RISC-V 汇编 0x0100FE7
4 lui s1, 0x12345 0x0100FE7
8 jalr t1, s1, 0x678 ... .. 0x1
2345678 ...
```

对于偏移量。偏移量的最高有效位编码在B型和J型指令的12位和20位立即数字段中。偏移量的最低有效位始终为0，因此不编码在指令中。auipc（将高位立即数加到PC）指令也使用PC相对寻址。例如，指令auipc s3, 0xABCDE将PC + 0xABCDE000存入s3。

6.4.7 解释机器语言代码

要解释机器语言，必须解读每条32位指令字中的字段。不同指令使用不同格式，但所有格式都共享一个7位操作码字段。因此，最佳起点是查看操作码，以确定它是R型、I型、S/B型还是U/J型指令。

示例6.6 将机器语言翻译为汇编语言

将以下机器语言代码翻译成汇编语言。

```
0x41FE83B3
0xFDA48293
```

解决方案 首先，我们将每条指令表示为二进制形式，并查看最低的七个有效位以找到每条指令的操作码。

```
0100 0001 1111 1110 1000 0011 1011 0011 (0x41FE83B3)
1111 1101 1010 0100 1000 0010 1001 0011 (0xFDA48293)
```

操作码决定了如何解释其余位。第一条指令的操作码是 0110011₂；因此，根据附录B中的表B.1，它是一个R型指令，我们可以将剩余位划分为R型字段，如图6.28顶部所示。第二条指令的操作码是 0010011₂，这意味着它是一个I型指令。我们将剩余位分组为I型格式，如图6.28所示，该图展示了这两条机器指令对应的汇编代码。

Machine Code							Field Values						Assembly	
							funct7	rs2	rs1	funct3	rd	op		
(0x41FE83B3)	0100 000	11111	11101	000	00111	011 0011	32	31	29	0	7	51	sub x7, x29,x31 sub t2, t4, t6	
	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits	7 bits	5 bits	5 bits	3 bits	5 bits	7 bits		
							funct7	rs2	rs1	funct3	rd	op		
(0xFDA48293)	1111 1101 1010	01001	000	00101	001 0011		-38	9	0	5	19		addi x5, x9, -38 addi t0, s1, -38	
	12 bits	5 bits	3 bits	5 bits	7 bits		12 bits	5 bits	3 bits	5 bits	7 bits			

图6.28 机器码到汇编代码的翻译

6.4.8 存储程序的力量

用机器语言编写的程序是一系列表示指令的32位数字。与其他二进制数一样，这些指令可以存储在内存中。这被称为*stored program*概念，也是计算机如此强大的关键原因。运行不同的程序无需花费大量时间和精力重新配置或重新连接硬件，只需将新程序写入内存即可。与专用硬件相比，存储程序提供了通用计算能力。通过这种方式，计算机只需更改存储的程序，就能执行从计算器到文字处理器再到视频播放器的各种应用程序。

存储程序中的指令从内存中检索（即*fetch*ed），并由处理器执行。即使是庞大复杂的程序，也只是一系列内存访问和指令执行。图 6.29展示了机器指令在内存中的存储方式。在RISC-V程序中，指令通常从低地址开始存储，但具体实现可能有所不同。图 6.29显示了存储在地址0x00000830至0x0000083C之间的代码。请记住，RISC-V内存是按字节寻址的，因此指令地址每次增加4，而非1。

为了运行或执行存储的程序，处理器从内存中顺序获取指令。获取的指令随后由数字硬件进行解码和执行。当前指令的地址保存在32位程序计数器（PC）寄存器中。

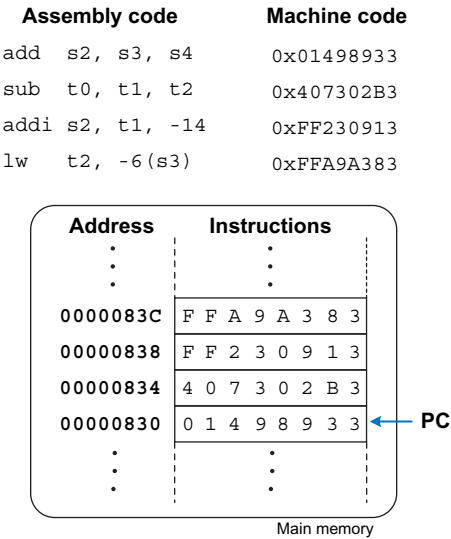


Figure 6.29 Stored program

为了执行图6.29中的代码，PC被初始化为地址0x00000830。处理器从该内存地址获取指令并执行指令0x01498933（add s2, s3, s4）。然后处理器将PC增加4至0x00000834，获取并执行该指令，并重复此过程。

微处理器的*architectural state*包含了决定程序行为所需的一切要素。对于RISC-V架构而言，其架构状态包括存储器、寄存器文件和程序计数器（PC）。如果操作系统（OS）在程序执行的某个时刻保存了架构状态，它就可以中断该程序、执行其他任务，然后恢复状态，使程序能够正确继续运行，仿佛从未被中断过。在第7章构建微处理器时，架构状态也具有极其重要的意义。

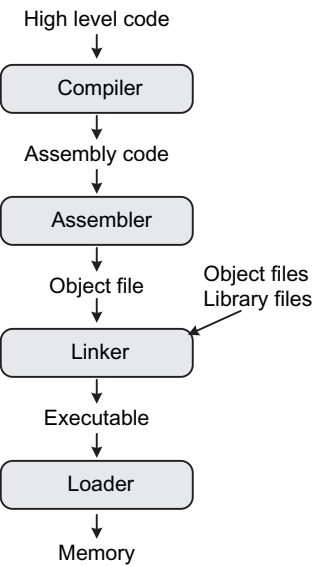


图6.30 翻译和启动程序的步骤

6.5 灯光、摄像、开拍：编译、汇编、且加载*

到目前为止，我们已经展示了如何将简短的高级代码片段翻译成汇编语言和机器码。本节将描述如何编译和汇编一个完整的高级程序，以及如何将程序加载到内存中执行。我们首先介绍一个示例RISC-V *memory map*，它定义了代码、数据和栈内存的位置。

图6.30展示了将高级语言程序翻译为机器语言并开始执行该程序所需的步骤。首先，*compiler*将高级代码翻译为汇编代码。*assembler*将汇编代码翻译为机器代码，并将其放入目标文件中。*linker*将机器代码与来自库和其他文件的代码合并，并确定正确的分支地址和变量位置，以生成完整的可执行程序。在实践中，大多数编译器会执行编译、汇编和链接这三个步骤。最后，*loader*将程序加载到内存中并开始执行。本节的剩余部分将通过一个简单程序逐步说明这些步骤。

6.5.1 内存映射

采用32位地址时，RISC-V地址空间可寻址 2^{32} 字节（4 GB）。字地址是4的倍数，范围从0到0xFFFFFFF。图6.31展示了一个示例内存映射。我们的内存映射将地址空间划分为五个部分或*segments*：文本段、全局数据段、动态数据段、异常处理程序段以及操作系统（OS）段（包含专用于输入/输出（I/O）的内存）。以下各节将描述每个段。我们给出一个

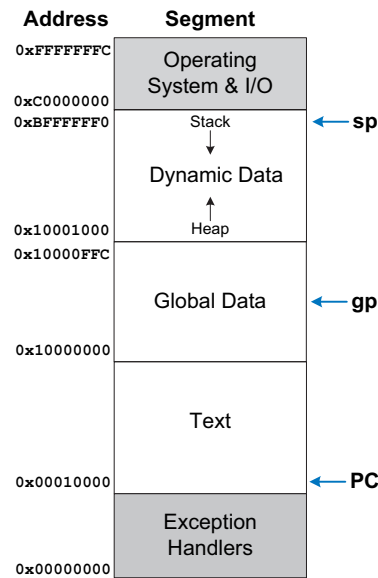


图6.31 RISC-V内存映射示例

示例 RISC-V 内存映射在此处；然而，RISC-V 并未定义特定的内存映射。尽管异常处理程序通常位于低地址或高地址，但用户可以定义文本（代码和常量数据）、内存映射 I/O、栈和全局数据的放置位置。这提供了灵活性，尤其是在较小的系统中，例如手持设备，其中仅使用部分内存范围，因此仅填充物理内存。

文本段

text segment 存储机器语言用户程序。除代码外，它还可能包含字面量（常量）和只读数据。

全局数据段

global data segment 存储全局变量，与局部变量不同，全局变量可被程序中所有函数访问。局部变量在函数内定义，仅能被该函数访问，通常位于寄存器或栈中。全局变量在程序开始执行前被分配在内存中，通常通过指向全局数据段中部的 *global pointer* 寄存器 *gp*（寄存器 x3）进行访问。在此例中，*gp* 为 0x10000800。利用 12 位有符号偏移量，程序员可通过 *gp* 访问整个全局数据段。

RISC-V要求sp保持16字节对齐，以便与操作128位（即16字节）数据的四精度RISC-V基础指令集RV128I兼容。因此，即使只需要较小的栈空间，sp也会按16的倍数递减以在栈上腾出空间。我们在第6.3.7节中略过了这一要求，以突出约定之上的功能。

动态数据段

*dynamic data segment*持有栈和堆。该段中的数据在启动时未知，而是在程序执行过程中动态分配和释放。

启动时，操作系统将栈指针（sp，寄存器x2）设置为指向栈顶，此处为0xBFFFFFF0。栈通常向下增长，如图所示。栈包含临时存储和局部变量（如数组），这些内容无法容纳在寄存器中。如第6.3.7节所述，函数也使用栈来保存和恢复寄存器。每个栈帧按后进先出的顺序访问。

heap 存储程序在运行时分配的数据。在 C 语言中，内存分配通过 `malloc` 函数完成；在 C++ 和 Java 中，则使用 `new` 来分配内存。就像宿舍地板上堆衣服一样，堆数据可以按任意顺序使用和丢弃。堆通常从动态数据段的底部向上增长。

如果栈和堆相互增长，程序的数据可能会被破坏。内存分配器通过在没有足够空间分配更多动态数据时返回内存不足错误，来确保这种情况永远不会发生。

异常处理器、操作系统和I/O段

示例RISC-V内存映射的最低部分保留给异常处理程序（见第6.6.2节）和启动时运行的引导代码。内存映射的最高部分保留给操作系统和内存映射I/O（见第9.2节）。

6.5.2 汇编器指令

汇编指令指导汇编器分配和初始化全局变量、定义常量以及区分代码和数据。表6.5列出了常见的RISC-V汇编指令，代码示例6.29展示了如何使用它们。

`.data`、`.text`、`.bss` 和 `.section .rodata` 汇编指令分别告诉汇编器将后续的数据或代码放置在内存的全局数据段、文本（代码）段、BSS 段或只读数据（`.rodata`）段中。BSS 段位于全局数据段内，但初始化为零。只读数据段是放置在文本段（即 *program memory* 中）的常量数据。

BSS stands for *block started symbol* and it was initially a keyword to allocate a block of uninitialized data. Now, most operating systems initialize data in the BSS segment to zero.

代码示例6.29中的程序首先将main标签设为全局(`globl main`)，以便main函数能够从该文件外部被调用，通常由操作系统或引导加载程序调用。随后将N的值设为5(`equ N, 5`)。汇编器在翻译前会将N替换为5。

表6.5 RISC-V汇编器指令

Assembler Directive	Description
<code>.text</code>	Text section
<code>.data</code>	Global data section
<code>.bss</code>	Global data initialized to 0
<code>.section .foo</code>	Section named <code>.foo</code>
<code>.align N</code>	Align next data/instruction on 2^N -byte boundary
<code>.balign N</code>	Align next data/instruction on N -byte boundary
<code>.globl sym</code>	Label <code>sym</code> is global
<code>.string "str"</code>	Store string <code>"str"</code> in memory
<code>.word w1, w2, ..., wN</code>	Store N 32-bit values in successive memory words
<code>.byte b1, b2, ..., bN</code>	Store N 8-bit values in successive memory bytes
<code>.space N</code>	Reserve N bytes to store variable
<code>.equ name, constant</code>	Define symbol <code>name</code> with value <code>constant</code>
<code>.end</code>	End of assembly code

将汇编指令转换为机器码。例如，指令 ``lw t5, N*4(t0)`` 被翻译为 ``lw t5, 20(t0)``，然后转换为机器码 (0x0142AF03)。接下来，程序分配以下全局变量，如图 6.32 所示：A（一个包含 7 个 32 字节元素的数组）、str1（一个以空字符结尾的字符串）、B 和 C（各 4 字节）以及 D（1 字节）。A、B 和 str1 分别初始化为 {5, 42, -88, 2, -5033, 720, 314}、0x32A 和 "RISC-V"（即 {52, 49, 53, 43, 2D, 56, 00}——参见表 6.2）。请记住，在 C 编程语言中，字符串以空字符 (0x00) 结尾。变量 C 和 D 未由用户初始化，位于 BSS 段中。该汇编器在数据段和 BSS 段之间包含 16 字节未分配的内存，如图 6.32 中的灰色框所示。

`.align 2` 汇编指令将后续数据或代码对齐到 $2^2 = 4$ 字节边界。`.balign 4`（字节对齐 4）汇编指令与之等效。这些汇编指令有助于维护数据和指令的对齐。例如，如果在分配 B 之前（即在 B: `.word 0x32A` 之前）移除 `.align 2`，那么 B 将直接分配在 str1 变量之后，位于字节 0x2157 – 0x215A（而不是 0x2158 – 0x215B）。

代码示例6.29中的程序已在西部数据开源商业SweRV EH1 RISC-V核心上运行。其他处理器使用不同的内存映射，因此会将变量和代码放置在不同的地址。Imagination Technologies提供的免费RVfpga（RISC-V FPGA）课程展示了如何将SweRV EH1核心部署到FPGA上运行C语言和汇编程序，并探索、扩展和修改该RISC-V处理器及系统。详见<https://university.imgtec.com/rvfpga/>。

注意，str2位于代码段（*not*数据段）中地址0x140处，靠近起始地址为0x88的用户代码（main）。通过将代码和数据放在一起，程序最小化了所需的内存以及访问数据所需的指令数量，这两者在手持设备和嵌入式系统中都至关重要。

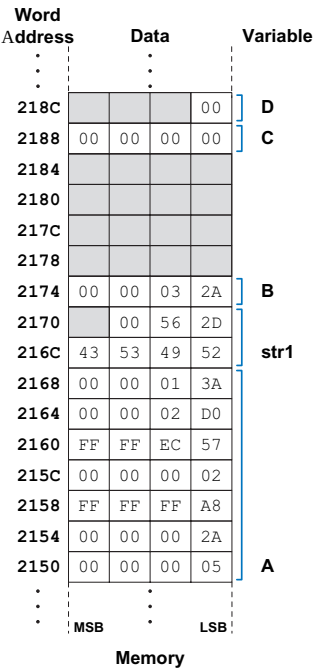


图6.32 代码示例6.29中全局变量的内存分配

代码示例6.29 使用汇编器指令

```
.globl main           # make the main label global
.equ N, 5              # N = 5

.data                 # global data segment
A: .word 5, 42, -88, 2, -5033, 720, 314
str1: .string "RISC-V"
.align 2              # align next data on 2^2-byte boundary
B: .word 0x32A

.bss                  # bss segment - variables initialized to 0
C: .space 4
D: .space 1

.balign 4             # align next instruction on 4-byte boundary
.text                 # text segment (code)
main:
    la t0, A           # t0 = address of A           = 0x2150
    la t1, str1         # t1 = address of str1        = 0x216C
    la t2, B            # t2 = address of B           = 0x2174
    la t3, C            # t3 = address of C           = 0x2188
    la t4, D            # t4 = address of D           = 0x218C
    lw t5, N*4(t0)      # t5 = A[N] = A[5] = 720      = 0x2D0
    lw t6, 0(t2)        # t6 = B = 810               = 0x32A
    add t5, t5, t6       # t5 = A[N] + C = 720 + 810 = 1530 = 0x5FA
    sw t5, 0(t3)        # C = 1530                  = 0x5FA
    lb t5, N-1(t1)      # t5 = str1[N-1] = str1[4] = '-' = 0x2D
    sb t5, 0(t4)        # D = str1[N-1]              = 0x2D
    la t5, str2          # t5 = address of str2       = 0x140
    lb t6, 8(t5)        # t6 = str2[8] = 'r'         = 0x72
    sb t6, 0(t1)        # str1[0] = 'r'              = 0x72
    jr ra               # return

.section .rodata
str2: .string "Hello world!"
.end                   # end of assembly file
```

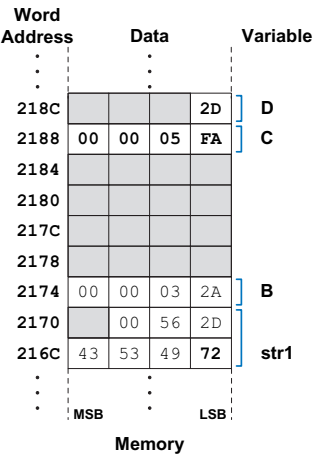


图6.33 全局变量C、D和str1的最终值

main函数首先通过加载地址（la）伪指令（见表B.7，位于教科书封面内侧）将全局变量的地址加载到t0–t4中。程序从内存中取出A[5]和C，将它们相加，并将结果（0x5FA）存入D。然后，它使用指令lb t5, N-1(t1)加载str1[4]的值（即‘-’ = ASCII码0x2D），并将该值存入全局变量B。最后，程序读取str2[8]（即字符‘r’），并将该值存入str1[0]。main函数通过jr ra返回操作系统或启动代码结束。图6.33显示了C、D和str1的最终值。end汇编指令表示汇编文件的结束。

6.5.3 编译

编译器将高级代码翻译成汇编语言，然后汇编器再将汇编代码翻译成机器码。本节中的示例基于GCC，这是一款流行且广泛使用的

免费编译器。GCC是*toolchain*的一部分，该集合包含其他功能，其中一些将在本节中讨论。代码示例6.30展示了一个简单的高级程序，包含三个全局变量和两个函数，以及由SiFive的Freedom E SDK工具链中的GCC生成的汇编代码。关于使用RISC-V编译器的说明，请参阅前言。

在代码示例6.30中，`main`函数首先将`ra`存储在栈上。它为四个字（16字节）预留了空间，但只使用了其中一个栈位置。请记住，`sp`必须保持16字节对齐，以便

代码示例6.30 编译高级程序

High-Level Code	RISC-V Assembly Code
<pre>int f, g, y; int func(int a, int b) { if (b < 0) return (a + b); else return(a + func(a, b - 1)); } void main() { f=2; g=3; y=func(f,g); return; }</pre>	<pre>.text .globl func .type func,@function func: addi sp,sp,-16 sw ra,12(sp) sw s0,8(sp) mv s0,a0 add a0,a1,a0 bge a1,zero,.L5 .L1: lw ra,12(sp) lw s0,8(sp) addi sp,sp,16 jr ra .L5: addi a1,a1,-1 mv a0,s0 call func add a0,a0,s0 j .L1 .globl main .type main,@function main: addi sp,sp,-16 sw ra,12(sp) lui a5,%hi(f) li a4,2 sw a4,%lo(f)(a5) lui a5,%hi(g) li a4,3 sw a4,%lo(g)(a5) li a1,3 li a0,2 call func lui a5,%hi(y) sw a0,%lo(y)(a5) lw ra,12(sp) addi sp,sp,16 jr ra .comm y,4,4 .comm g,4,4 .comm f,4,4</pre>



格蕾丝·霍珀，1906–1902
毕业于耶鲁大学，获得数学博士学位。在雷明顿-兰德公司工作期间开发了首个编译器，并对COBOL编程语言的开发起到了关键作用。作为一名海军军官，她获得了多项荣誉，包括二战胜利勋章和国防服役勋章。她还记录了第一个计算机“bug”，而这里的“bug”实际上是一只干扰打孔卡的昆虫。

与RV128I兼容。main随后将值2写入全局变量f，将值3写入全局变量g。这些全局变量尚未放入内存——后续将由汇编器处理。因此，目前汇编代码对每个全局变量的存储使用两条指令（lui后接sw），而非仅用一条指令（sw），以防需要指定32位地址。

程序随后将f和g（即2和3）放入参数寄存器a0和a1中，并通过伪代码call func调用func。函数func将ra和s0保存在栈上。接着，它将a0（a）存入s0（因为递归调用func后需要用到它），并计算 $a0 = a0 + a1$ （返回值= $a + b$ ）。然后，func根据a1（b）是否大于等于零进行分支。否则，它恢复ra、s0和sp，并通过jr ra返回。如果分支成立（ $b \geq 0$ ），func递减a1（b），并递归调用func。从递归调用返回后，它将返回值（a0）与s0（a）相加，并跳转到标签.L1，在那里恢复ra、s0和sp，函数返回。主

该函数将func(a0)的返回结果存储到全局变量y中，恢复ra和sp，并返回y。在汇编代码底部，程序通过.comm g, 4, 4等指令表明它有三个4字节宽的全局变量f、g和y。第一个4表示4字节对齐，第二个4表示变量的大小（4字节）。

使用GCC编译、汇编和链接名为prog.c的C程序，命令如下：

```
gcc -O1 -g prog.c -o prog
```

该命令生成一个名为prog的可执行输出文件。-O1标志要求编译器执行基本优化，而不是生成效率极低的代码。-g标志告诉编译器在文件中包含调试信息。

要查看中间步骤，我们可以使用GCC的-S标志进行编译，但不进行汇编或链接。

```
gcc -O1 -S prog.c -o prog.s
```

输出文件prog.s相当冗长，但有趣的部分如代码示例6.30所示。

6.5.4 组装

assembler将汇编语言代码转换为包含机器语言代码的object file。GCC可以通过以下方式从prog.s或直接从prog.c创建目标文件：

```
gcc -c prog.s -o prog.o
```

或

```
gcc -O1 -g -c prog.c -o prog.o
```

汇编器对汇编代码进行两遍扫描。在第一遍扫描中，汇编器分配指令地址并查找所有符号，例如标签和全局变量名。符号的名称和地址保存在符号表中。在第二遍扫描代码时，汇编器生成机器语言代码。标签的地址从符号表中获取。机器语言代码和符号表存储在目标文件中。

我们可以使用objdump命令*disassemble*目标文件，以查看机器语言代码旁边的汇编语言代码。

```
objdump -S prog.o
```

以下展示了.text节的反汇编结果。如果代码最初使用-g选项编译，反汇编器还会显示对应的C代码行（如下所示），并与汇编代码交错排列。请注意，call伪指令被翻译成了两条RISC-V指令：auipc ra, 0x0和jalr ra，这是为了应对函数距离较远的情况，即函数距离当前PC的距离超过了jal指令有符号21位偏移量所能覆盖的范围。用于存储全局变量的指令在全局变量被放入内存之前也只是占位符。例如，地址0x48到0x50处的三条指令用于将值2存储到全局变量f中。一旦f在链接阶段被放入内存，这些指令将被更新。

00000000 <函数>:

```
int f, g, y; int func(int a, int b) { 0: ff010113 addi sp,sp,-16 4: 00
112623 sw ra,12(sp) 8: 00812423 sw s0,8(sp) c: 00050413 mv
s0,a0 if (b<0) return (a+b); 10: 00a58533 add a0,a1,a0 14: 0005
da63 bgez a1,28 <.L5> 00000018 <.L1>: else return(a + func(a, b-1
)); } 18: 00c12083 lw ra,12(sp) 1c: 00812403 lw s0,8(sp) 20: 010
10113 addi sp,sp,16 24: 00008067 ret 00000028 <.L5>: else retur
n(a + func(a, b-1)); 28: fff58593 addi a1,a1,-1 2c: 00040513 mv
a0,s0
```



```

30: 00000097          auipc ra,0x0
34: 000080e7          jalr  ra # 30 <.LVL5+0x4>
38: 00850533          add   a0,a0,s0
3c: fddff06f          j      18 <.L1>

00000040 <main>:
void main() {
40: ff010113          addi  sp,sp,-16
44: 00112623          sw    ra,12(sp)
f=2;
48: 000007b7          lui   a5,0x0
4c: 00200713          li    a4,2
50: 00e7a023          sw    a4,0(a5) # 0 <func>
g=3;
54: 000007b7          lui   a5,0x0
58: 00300713          li    a4,3
5c: 00e7a023          sw    a4,0(a5) # 0 <func>
y=func(f,g);
60: 00300593          li    a1,3
64: 00200513          li    a0,2
68: 00000097          auipc ra,0x0
6c: 000080e7          jalr  ra # 68 <main+0x28>
70: 000007b7          lui   a5,0x0
74: 00a7a023          sw    a0,0(a5) # 0 <func>
return;
}
78: 00c12083          lw    ra,12(sp)
7c: 01010113          addi  sp,sp,16
80: 00008067          ret

```

我们可以使用 `objdump` 命令并加上 `-t` 标志来查看目标文件中的符号表。以下展示了其中有趣的部分。我们为三个感兴趣的列添加了标签：符号的内存地址、大小和名称。由于程序尚未被加载到内存中（尚未链接），这些地址目前只是占位符。`.text` 表示代码（文本）段，`.data` 表示数据（全局数据）段。这两个符号的大小目前为 0，因为程序尚未被链接。两个函数 `func` 和 `main` 的大小被列出：`func` 为 `0x40`（64）字节 = 16 条指令，`main` 为 `0x44`（68）字节 = 17 条指令，如上文代码所示。全局变量符号 `f`、`g` 和 `y` 也被列出，每个大小为 4 字节，但它们的地址被列为占位符值 `0x00000004`，因为它们尚未被分配地址。

```
objdump -t prog.o
```

符号表:

Address		Size	Symbol Name
00000000	l d .text	00000000	.text
00000000	l d .data	00000000	.data
00000000	g F .text	00000040	func
00000040	g F .text	00000044	main
00000004	0 *COM*	00000004	f
00000004	0 *COM*	00000004	g
00000004	0 *COM*	00000004	y

在此符号表中，我们将在很大程度上忽略未标记的列。这些列显示了与符号关联的标志（l表示局部或g表示全局，d表示调试，F表示函数，或O表示对象）以及符号所在的节（.text、.data 或 *COM*（通用），当它不在某个节中时）。

6.5.5 链接

大多数大型程序都包含多个文件。如果程序员只编辑了其中一个文件，重新编译和汇编其他文件将造成浪费。特别是，程序经常调用库文件中的函数；这些库文件几乎从不更改。如果高级代码文件未被修改，则无需更新相应的目标文件。此外，程序通常包含一些启动代码（用于初始化堆栈、堆等），这些代码必须在调用主函数之前执行。

链接器的工作是将所有目标文件和启动代码合并成一个名为 *executable* 的机器语言文件，并为全局变量分配地址。链接器会重新定位目标文件中的数据和指令，使它们不会相互重叠。它利用符号表中的信息，根据新的标签和全局变量地址调整代码。调用GCC来链接目标文件，使用以下命令：

```
gcc prog.o -o prog
```

我们可以再次使用以下命令反汇编可执行文件：

```
objdump -S -t prog
```

启动代码过于冗长，此处不再展示，但以下是从可执行文件中反汇编得到的更新后的符号表和程序代码。我们再次为感兴趣的列添加了标签。函数和全局变量现已重定位至实际地址。根据符号表，整个文本段和数据段（包括启动代码和系统数据）分别起始于0x10074和0x115e0。func函数起始地址为0x10144，长度为0x3c字节（15条指令）。main函数起始地址为0x10180，长度为0x34字节（13条指令）。全局变量各占4字节；f位于内存地址0x11a30，g位于0x11a34，y位于0x11a38。

注意，现在全局变量 f、g 和 y 已被分配内存地址，它们被列为全局符号（由 g 标志指示），并位于 .bss 段中，该段用于存放未初始化的全局变量。

符号表：

Address	Size	Symbol Name
00010074 l d .text	00000000	.text
000115e0 l d .data	00000000	.data
00010144 g F .text	0000003c	func
00010180 g F .text	00000034	main
00011a30 g 0 .bss	00000004	f
00011a34 g 0 .bss	00000004	g
00011a38 g 0 .bss	00000004	y

注意，如下所示，func的大小现在只有15条指令，而非16条。对func的调用是近调用；因此，只需一条指令jalr即可完成调用。同样，由于近调用和存储位置靠近全局指针gp，主代码从17条指令减少到13条。程序使用单条指令sw a4, -944(gp)将数据存储到f。通过这条指令，我们还可以确定由启动代码初始化的全局指针gp的值。我们知道f的地址是0x11a30；因此，gp = 0x11a30 + 944 = 0x11DE0。

00010144 <func>: int f,
g, y;

```
int func(int a, int b) { 10144: ff010113 addi sp,sp,-16 10148: 00112623 sw
ra,12(sp) 1014c: 00812423 sw s0,8(sp) 10150: 00050413 mv s0,a0 if (b<0)
return (a+b); 10154: 00a58533 add a0,a1,a0 10158: 0005da63 bgez a1,1016
c <func+0x28> else return(a + func(a, b-1)); } 1015c: 00c12083 lw ra,12(sp)
10160: 00812403 lw s0,8(sp) 10164: 01010113 addi sp,sp,16 10168: 00008
067 ret else return(a + func(a, b-1)); 1016c: fff58593 addi a1,a1,-1 10170:
00040513 mv a0,s0 10174: fd1ff0ef jal ra,10144 <func> 10178: 00850533
add a0,a0,s0 1017c: fe1ff06f j 1015c <func+0x18>
```

00010180 <主程序>:

```
void main() {
    10180: ff010113      addi  sp,sp,-16
    10184: 00112623      sw   ra,12(sp)
    f=2;
    10188: 00200713      li   a4,2
    1018c: c4e1a823      sw   a4,-944(gp) # 11a30 <f>
    g=3;
    10190: 00300713      li   a4,3
    10194: c4e1aa23      sw   a4,-940(gp) # 11a34 <g>
    y=func(f,g);
    10198: 00300593      li   a1,3
    1019c: 00200513      li   a0,2
    101a0: fa5ff0ef      jal  ra,10144 <func>
    101a4: c4a1ac23      sw   a0,-936(gp) # 11a38 <y>

    return;
}
    101a8: 00c12083      lw   ra,12(sp)
    101ac: 01010113      addi  sp,sp,16
    101b0: 00008067      ret
```

6.5.6 加载中

操作系统通过从存储设备（通常是硬盘或闪存）读取可执行文件的文本段，将其加载到内存的文本段中。操作系统跳转到程序起始位置开始执行。图6.34展示了程序执行开始时的内存映射。

6.6 零碎事项*

本节涵盖了一些可选主题，这些主题在本章其他部分中自然无法容纳。这些主题包括字节序、异常、有符号和无符号算术指令、浮点指令以及压缩（16位）指令。

6.6.1 字节序

按字节寻址的存储器采用大端或小端方式组织，如图6.35所示。在这两种格式中，32位字的最高有效字节（MSB）位于左侧，最低有效字节（LSB）位于右侧。两种格式的字地址相同，且指向相同的四个字节。只有字内字节的地址不同（见图6.35）。在*big-endian*机器中，字节从大端（最高有效）开始编号为0。在*little-endian*机器中，字节从小端（最低有效）开始编号为0。

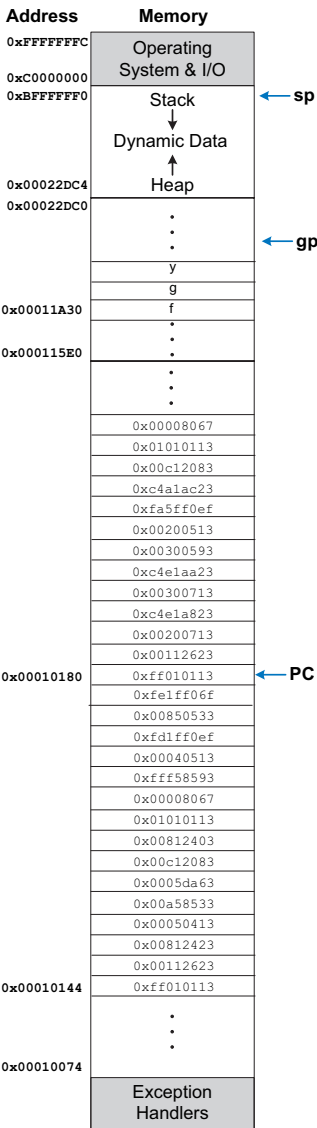
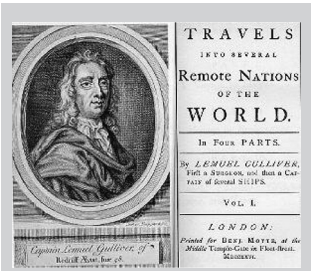


图6.34 程序加载到内存中



术语big-endian和little-endian源自乔纳森·斯威夫特的Gulliver's，该书于1726年首次以艾萨克·比克斯塔夫的笔名出版。在他的故事中，小人国国王要求其臣民（小端派）从小端敲破鸡蛋。而大端派则是从大端敲破鸡蛋的叛乱者。

这些术语最早由丹尼·科恩（Danny Cohen）在1980年愚人节发表的论文《论圣战与和平恳求》（USC/ISI IEN）中应用于计算机架构。（照片由利兹大学图书馆布罗瑟顿收藏提供。）

存在第四种特权级别，称为hypervisor mode（H-mode），它支持machine virtualization，即在一台物理机器上运行多个机器（可能具有多个操作系统）的外观。H模式的特权高于S模式，但低于M模式。

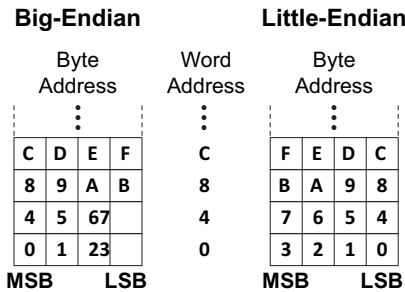


图6.35 大端与小端内存寻址

RISC-V 通常采用小端序，尽管也定义了大端序的变体。IBM 的 PowerPC（曾用于 Macintosh 计算机）使用大端序寻址。Intel 的 x86 架构（用于 PC）使用小端序寻址。端序的选择完全是任意的，但在大端序和小端序计算机之间共享数据时会带来麻烦。在本文的示例中，每当字节顺序重要时，我们都使用小端序格式。

6.6.2 例外情况

exception 类似于由硬件或软件中的事件引起的非计划函数调用。例如，处理器可能收到用户按下键盘按键的通知。处理器可能会暂停当前操作，确定按下了哪个键，将其保存以供后续参考，然后恢复正在运行的程序。这种由输入/输出（I/O）设备（如键盘）触发的硬件异常通常被称为interrupt。另一种情况是，程序可能遇到由软件引起的错误条件，例如未定义的指令。软件异常有时被称为traps。异常的其他原因包括复位以及尝试读取不存在的内存。与任何其他函数调用一样，异常必须保存返回地址，跳转到某个地址，执行其任务，清理自身，然后返回到程序中断处继续执行。

执行模式与特权级别
一个RISC-V处理器可以在具有不同特权级别的几种执行模式之一中运行。特权级别决定了可以执行哪些指令以及可以访问哪些内存。RISC-V的三个主要特权级别按特权递增顺序为用户模式、supervisor mode和machine mode。机器模式（M模式）是最高特权级别；在此模式下运行的程序可以访问所有寄存器和内存位置。M模式是唯一必需的特权模式，也是唯一使用的模式。

在没有操作系统（OS）的处理器中，包括许多嵌入式系统。运行在操作系统之上的用户应用程序通常运行在用户模式（U-mode）下，而操作系统运行在监管模式（S-mode）下；用户程序无法访问为操作系统保留的特权寄存器或内存位置。不同的模式可以防止关键状态被破坏。我们讨论在机器模式（M-mode）下运行时的异常情况。在其他级别发生的异常类似，但使用与该模式相关的寄存器。

异常处理器
异常处理程序使用四个专用寄存器，称为*control and status registers*（CSRs），来处理异常：mtvec、mcause、mepc和mscratch。机器陷阱向量基地址寄存器mtvec保存异常处理程序代码的地址。当异常发生时，处理器将异常原因记录在mcause中（见表6.6），将异常指令的PC存储在机器异常PC寄存器mepc中，并跳转到mtvec中预配置的地址处的异常处理程序。

跳转到mtvec中的地址后，异常处理程序读取mcause寄存器以检查异常原因并做出相应响应（例如，在硬件中断时读取键盘）。

表6.6 常见异常原因编码

Interrupt	Exception Code	Description
1	3	Machine software interrupt
1	7	Machine timer interrupt
1	11	Machine external interrupt
0	0	Instruction address misaligned
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store address misaligned
0	7	Store access fault
0	8	Environment call from U-Mode
0	9	Environment call from S-Mode
0	11	Environment call from M-Mode

RISC-V定义了大量CSR，所有这些都必须在启动时初始化。

mcause的值可以分为中断或异常，如表6.6最左列所示，该列为mcause的位31。mcause的[30:0]位保存*exception code*，指示中断或异常的原因。

异常处理可采用两种模式之一：直接模式或向量模式。RISC-V通常使用此处描述的直接模式，即所有异常都跳转到同一地址，该地址为mtvec寄存器中编码在[31:2]位的基础地址。在向量模式下，异常会根据触发原因从基础地址偏移*offset*个地址单元跳转。每个偏移量之间间隔少量地址（例如32字节），因此异常处理代码可能需要跳转到更大的异常处理程序来处理该异常。异常模式编码在mtvec寄存器的[1:0]位中：00₂表示直接模式，01₂表示向量模式。

异常相关寄存器特定于操作模式。M模式寄存器包括mtvec、mepc、mcause和mscratch，S模式寄存器包括sepc、scause和sscratch。H模式也有其自身的寄存器。为每种模式专用的独立异常寄存器为多个特权级别提供了硬件支持。

csrrw 是一条实际的 RISC-V 指令（参见附录 B 中的表 B.8），但 csrr 和 csw 是伪指令。csrr 实现为 csrrs rd, CSR, x0，而 csw 实现为 csrrw x0, CSR, rs1。

随后，程序要么中止，要么通过执行机器异常返回指令 `mret` 返回程序，该指令跳转到 `mepc` 中的地址。将异常指令的 PC 保存在 `mepc` 中，类似于在 `jal` 指令中使用 `ra` 存储返回地址。异常处理程序必须使用程序寄存器（x1-x31）来处理异常，因此它们利用 `mscratch` 指向的内存来保存和恢复这些寄存器。

异常相关指令

异常处理程序使用特殊指令来处理异常。这些指令被称为 *privileged instructions*，因为它们访问CSR。它们是基础RV32I指令集的一部分（参见附录B，表B.8）。mepc和mcause寄存器不属于RISC-V程序寄存器（x1-x31），因此异常处理程序必须将这些特殊用途（CSR）寄存器移入程序寄存器以读取和操作它们。RISC-V使用三条指令来读取、写入或同时读取和写入CSR：csrr（读取CSR）、csw（写入CSR）和csrrw（读取/写入CSR）。例如，csrr t1, mcause将mcause中的值读入t1；csw mepc, t2将t2中的值写入mepc；而csrrw t1, mscratch, t0同时将mscratch中的值读入t1并将t0中的值写入mscratch。

异常处理总结

总之，当处理器检测到异常时，它会：

- 1. 跳转到 mtvec 中保存的异常处理程序地址。
- 2. 异常处理程序将寄存器保存到由 mscratch 指向的小型栈上，然后使用 csrr（读取 CSR）查看异常原因（编码在 mcause 中）并做出相应响应。
- 3. 处理程序完成后，可选择将 mepc 增加 4，从内存中恢复寄存器，然后中止程序或使用 mret 指令返回用户代码，该指令会跳转到 mepc 中保存的地址。

示例6.7 异常处理程序代码

编写一个异常处理程序，用于处理以下两种异常：非法指令（mcause = 2）和加载地址未对齐（mcause = 4）。如果发生非法指令异常，程序应直接继续执行该非法指令之后的指令。如果发生加载地址未对齐异常，程序应终止。如果发生任何其他异常，程序应尝试重新执行该指令。

解决方案 异常处理程序首先保存将被覆盖的程序寄存器。然后检查每个异常原因，并(1)在非法指令异常时，继续执行紧接在异常指令之后的位置（即mepc + 4），(2)在加载地址未对齐时中止，或(3)在其他异常情况下尝试重新执行异常指令（即返回到mepc）。在返回程序之前，处理程序恢复所有被覆盖的寄存器。为了中止程序，处理程序跳转到位于exit标签处的退出代码（未显示）。对于在操作系统上运行的程序，jexit指令可以被环境调用（ecall）替代，返回码存储在程序寄存器（如a0）中。

```
# save registers that will be overwritten
csrrw t0, mscratch, t0    # swap t0 and mscratch
sw     t1, 0(t0)          # save t1 on mscratch stack
sw     t2, 4(t0)          # save t2 on mscratch stack

# check cause of exception
csrr   t1, mcause         # t1 = mcause
addi   t2, x0, 2          # t2 = 2 (illegal instruction exception code)

illegalinstr:
bne    t1, t2, checkother # branch if not an illegal instruction
csrr   t2, mepc           # t2 = exception PC
addi   t2, t2, 4          # increment exception PC by 4
csrw   mepc, t2           # mepc = mepc + 4
j      done               # restore registers and return

checkother:
addi   t2, x0, 4          # t2 = 4 (misaligned load exception code)
bne    t1, t2, done       # branch if not a misaligned load
j      exit               # exit program

# restore registers and return from the exception
done:
lw     t1, 0(t0)          # restore t1 from mscratch stack
lw     t2, 4(t0)          # restore t2 from mscratch stack
csrrw  t0, mscratch, t0   # swap t0 and mscratch
mret                   # return to program (PC = mepc)
...
exit:
...
```

启动时，处理器跳转到 *reset exception*，这是一个硬连线内存地址——例如0x200——即 *boot loader* 代码的起始地址，也称为 *boot code*。虽然复位并非程序执行过程中典型的异常，但由于复位是处理器的异常状态，因此仍被称为异常。引导代码配置内存系统，初始化CSR和栈指针，并从磁盘读取部分操作系统。随后，它启动操作系统中更长的引导过程。操作系统最终将加载程序，切换到非特权用户模式，并跳转到程序的起始位置。在 *bare metal* 系统（即无操作系统的系统）中，用户代码（可能包含用于设置栈指针等的轻量级引导代码）通常直接放置在复位向量处。

一个特别重要的例外是 *system call*，也称为 *environment*。程序使用这些来调用操作系统中的函数，该函数以比用户代码更高的特权级别运行。此例外由执行ecall指令的用户程序发起。与函数调用类似，程序在进行系统调用之前可以设置参数寄存器。

与其他架构（如MIPS和ARM）不同，RISC-V不包含用于检测溢出的指令（或异常），因为可以通过一系列现有指令来检测溢出。例如，以下代码检测t1和t2相加时的无符号溢出。

```
add t0, t1, t2 bltu t0, t1, overflow
```

换句话说，如果结果（t0）小于任一操作数（在此情况下为t1），则发生了溢出。

以下代码检测两个有符号数t1和t2相加时的溢出：

```
add t0, t1, t2 slti t3, t2, 0 slt t4, t0, t1 bne t3, t4, overflow
```

以方程形式表示，溢出 = $(t2 < 0) \& (t0 \geq t1) \vee (t2 \geq 0) \& (t0 < t1)$

换言之，当操作数为负（t3 = 1）且结果不小于另一操作数（t4 = 0），或当操作数大于或等于0（t3 = 0）且结果小于另一操作数（t4 = 1）时，发生溢出。

6.6.3 有符号与无符号指令

回想一下，二进制数可以有符号或无符号的。与大多数架构一样，RISC-V对有符号数采用补码表示。RISC-V的某些指令分为有符号和无符号两种版本，包括乘法、除法、小于则置位、分支以及部分字加载。

乘法和除法

有符号数和无符号数的乘法和除法行为不同。例如，作为无符号数，0xFFFFFFFF表示一个很大的数，但作为有符号数，它表示-1。因此，如果数字是无符号的，0xFFFFFFFF × 0xFFFFFFFF的结果将是0xFFFFFFFFE0000001，但如果数字是有符号的，结果将是0x0000000000000001。（注意，有符号乘法和无符号乘法的低32位是相同的。）因此，乘法和除法分为有符号和无符号两种形式。mulh和div将操作数视为有符号数。mulhu和divu将操作数视为无符号数。mulhsu将第一个操作数视为有符号数，第二个操作数视为无符号数。所有乘法高位指令（mulh、mulhu和mulhsu）将最高有效的32位放入目标寄存器rd。结果的低32位对于无符号或有符号乘法是相同的，因此mul将乘法结果的低32位放入rd，适用于无符号和有符号乘法。

小于设置

*Set less than*指令比较两个寄存器（slt）或一个寄存器与一个立即数（slti）。小于比较也有有符号（slt和slti）和无符号（sltu和sltiu）版本。在有符号比较中，0x80000000小于任何其他数，因为它是最负的二进制补码数。在无符号比较中，0x80000000大于0x7FFFFFFF但小于0x80000001，因为所有数都是正数。注意，sltiu在将12位立即数视为无符号数之前会对其进行符号扩展。例如，sltiu s0, s1, {v*}1273将s1与0xFFFFFB07进行比较，将该立即数视为一个大的正数。

分支

*branch if less than*和*branch if greater than or equal to*指令也有有符号（blt和bge）和无符号（bltu和bgeu）版本。有符号版本将两个源操作数视为二进制补码数，而无符号版本则将源操作数视为无符号数。

载荷

如第6.3.6节所述，字节加载分为有符号（lb）和无符号（lbu）版本。lb将字节进行符号扩展，而lbu将字节进行零扩展以填满整个32位寄存器。类似地，有符号和无符号半字加载（lh和lhu）将两个字节加载到低半部分，并对字的高半部分进行符号扩展或零扩展。

6.6.4 浮点指令

RISC-V架构定义了可选的浮点扩展，分别称为RVF、RVD和RVQ，用于操作单精度、双精度和四精度浮点数。RVF/D/Q定义了32个浮点寄存器f0至f31，其宽度分别为32、64或128位。当处理器实现多个浮点扩展时，它使用浮点寄存器的低位部分来执行低精度指令。f0至f31与程序（也称为*integer*）寄存器x0至x31是分开的。与程序寄存器一样，浮点寄存器按照惯例保留用于特定用途，如表6.7所示。

附录B中的表B.3列出了所有浮点指令。计算和比较指令对所有精度使用相同的助记符，并在末尾附加.s、.d或.q来表示精度。例如，fadd.s、fadd.d和fadd.q分别执行单精度、双精度和四精度加法。其他浮点指令包括fsub、fmul、fdiv、fsqrt、fmadd（乘加）和fmin。内存访问对每种精度使用单独的指令。加载指令为flw、fld和flq，存储指令为fsw、fsd和fsq。

浮点指令使用R型、I型和S型格式，以及一种新格式——R4型指令格式（见附录B中的图B.1）。

表6.7 RISC-V浮点寄存器集

Name	Register Number	Use
ft0–7	f0–7	Temporary variables
fs0–1	f8–9	Saved variables
fa0–1	f10–11	Function arguments/Return values
fa2–7	f12–17	Function arguments
fs2–11	f18–27	Saved variables
ft8–11	f28–31	Temporary variables

代码示例6.31 使用for循环访问浮点数数组

High-Level Code	RISC-V Assembly Code
<pre>int i; float scores[200]; for (i = 0; i < 200; i = i + 1) scores[i] = scores[i] + 10;</pre>	<pre># s0 = scores base address, s1 = i addi s1, zero, 0 # i = 0 addi t2, zero, 200 # t2 = 200 addi t3, zero, 10 # t3 = 10 fcvt.s.w ft0, t3 # ft0 = 10.0 for: bge s1, t2, done # if i >= 200 then done slli t3, s1, 2 # t3 = i * 4 add t3, t3, s0 # address of scores[i] flw ft1, 0(t3) # ft1 = scores[i] fadd.s ft1, ft1, ft0 # ft1 = scores[i] + 10 fsw ft1, 0(t3) # scores[i] = t1 addi s1, s1, 1 # i = i + 1 j for # repeat done:</pre>

乘加指令需要这种格式，它使用四个寄存器操作数。代码示例6.31修改了代码示例6.21，使其对单精度浮点分数数组进行操作。更改部分以粗体显示。

6.6.5 压缩指令

RISC-V的压缩指令扩展（RVC）通过缩小控制字段、立即数字段和寄存器字段的大小，并利用冗余或隐含寄存器，将常用整数和浮点指令的大小缩减至16位。这种指令尺寸的减小降低了成本、功耗和所需内存——所有这些对于手持和移动应用都至关重要。根据RISC-V *Instruction Set Manual*，程序中通常有50%到60%的指令可以被RVC指令替代。16位指令仍基于基础指令集所确定的基础数据尺寸（32位、64位或128位）进行操作。如果处理器能够同时处理压缩指令和32位指令，汇编程序可以混合使用这两种指令。

大多数RV32I指令都有对应的压缩版本，以c.开头，如附录B的表B.6所列。为减小体积，大多数压缩指令仅指定两个寄存器；第一个源寄存器同时也是目标寄存器。多数压缩指令使用3位寄存器编码，从x8到x15这8个寄存器中指定一个。x8编码为000₂，x9编码为001₂，依此类推。立即数也更短（6–11位），且操作码可用的位数更少。图B.2（位于本书封底内侧）展示了压缩指令的格式。

代码示例6.32修改了代码示例6.21，以使用压缩指令。注意，常量200太大，无法放入压缩立即数中，因此使用非压缩的addi来初始化s0。由于没有压缩的c.bge，我们也使用了非压缩的bge。我们还递增s0作为指向scores[i]的指针，因为使用双操作数压缩指令进行移位和加法较为繁琐。程序从40字节缩减到22字节。

许多RISC-V汇编器会混合使用压缩和非压缩指令生成代码，尽可能采用压缩指令以最小化代码体积。

代码示例6.32 使用压缩指令

High-Level Code	RISC-V Assembly Code
<pre>int i; int scores[200]; for (i = 0; i < 200; i = i + 1) scores[i] = scores[i] + 10;</pre>	<pre># s0 = scores base address, s1 = i c.li s1, 0 # i = 0 addi t2, zero, 200 # t2 = 200 for: bge s1, t2, done # if i >= 200 then done c.lw a3, 0(s0) # a3 = scores[i] c.addi a3, 10 # a3 = scores[i] + 10 c.sw a3, 0(s0) # scores[i] = a3 c.addi s0, 4 # next element of # scores c.addi s1, 1 # i = i + 1 c.j for # repeat done:</pre>

6.7 RISC-V架构的演进

RISC-V被设计为一种商业可行、开源且稳健、高效、灵活的计算机架构。RISC-V与其他架构的区别在于其开源特性，采用基础指令集以简化兼容性，支持从嵌入式系统到高性能计算机的全范围微架构，提供既定义又可定制的扩展，并具备压缩指令和RV128I等功能，以优化硬件并支持现有及未来设计，确保该架构的长期生命力。

RISC-V还通过成立RISC-V国际组织（见riscv.org）创建了一个由工业界和学术界合作伙伴组成的社区，从而加速了创新、商业化和协作。该开发者联盟还帮助设计和批准RISC-V架构。截至2021年，RISC-V国际组织已发展到拥有超过500个工业界和学术界成员，包括西部数据、英伟达、微芯科技和三星。

RISC-V架构在The RISC-V (riscv.org/specifications) 中有描述。该手册的早期版本（直至2.2版）是一份珍贵的规范，简洁、易读，并穿插了架构中设计决策背后的原理。

6.7.1 RISC-V基础指令集与扩展

RISC-V包含多种基础指令集和扩展，可支持从手持设备中的小型廉价嵌入式处理器到高性能、多核、多线程系统等广泛处理器。RISC-V拥有32位、64位和128位基础指令集：RV32I/E、RV64I和RV128I。32位基础指令集包括本章讨论的标准版本RV32I，以及仅含16个寄存器、面向极低成本处理器的嵌入式版本RV32E。截至2021年，仅RV32I和RV64I指令集已冻结；RV32E和RV128I仍在定义中。除这些基础架构外，RISC-V还定义了表6.8所列的扩展。最常用的扩展——浮点运算（RVF/D/Q）、压缩指令（RVC）和原子指令（RVA）——已完全规范并冻结，以支持开发与商业化。其余扩展仍在开发中。

所有RISC-V处理器必须支持一种基础架构——RV32/64/128I或RV32E——并可选择支持扩展，例如压缩或浮点扩展。通过使用扩展而非新的架构版本，RISC-V减轻了微架构之间向后或向前兼容的负担。所有

表6.8 RISC-V扩展

Extension	Description	Status
M	Integer multiplication and division	Frozen
F	Single-precision floating-point	Frozen
D	Double-precision floating-point	Frozen
Q	Quad-precision floating-point	Frozen
C	Compressed instructions	Frozen
A	Atomic instructions	Frozen
B	Bit manipulations	Open
L	Decimal floating-point	Open
J	Dynamically translated languages	Open
T	Transactional memory	Open
P	Packed-SIMD instructions	Open
V	Vector operations	Open

处理器必须至少支持基础架构。然而，处理器无需支持所有（甚至任何）扩展。

要理解RISC-V架构的演进，必须先了解其之前的其他架构，尤其是MIPS架构。RISC-V遵循了MIPS架构的许多原则，但也从现代架构和应用的角度受益，加入了支持嵌入式、多核、多线程系统及可扩展性等特性。下一节将对RISC-V与MIPS架构进行比较。

6.7.2 RISC-V与MIPS架构的比较

RISC-V架构与John Hennessy在20世纪80年代开发的MIPS架构有许多相似之处，但它消除了一些不必要的复杂性——并在立即数方面引入了一些复杂性！相似之处包括汇编和机器码格式、指令助记符、寄存器命名以及栈和调用约定。不同之处包括RISC-V的立即数大小和编码、相对于PC（而非PC+4）的分支、分支和跳转均为PC相对寻址、去除了MIPS中的分支延迟槽、对源寄存器和目标寄存器指令字段的严格定义、临时寄存器、保存寄存器和参数寄存器的数量不同，以及通过在指令中包含更多控制位而具有更强的可扩展性。通过将rs1、rs2和rd保留在使用它们的每种指令类型的相同位域中，RISC-V相对于MIPS简化解码器硬件。类似地，RISC-V的立即数编码简化了立即数扩展硬件。

6.7.3 RISC-V与ARM架构的比较

ARM是一种RISC架构，与20世纪80年代的MIPS架构大致同期开发。在过去十多年间，ARM处理器主导了移动设备领域，并广泛应用于机器人、弹球机及服务器等其他场景。ARM与RISC-V的相似之处包括：机器码格式和汇编指令数量较少，以及类似的堆栈与调用约定。ARM与RISC-V的不同之处在于：支持条件执行、复杂的内存访问索引模式、通过单条指令实现多寄存器压栈/出栈、可选的源寄存器移位操作，以及非常规的立即数编码方式。立即数编码采用8位数值加4位循环移位，且仅编码正立即数（减法由控制位决定）。这些特性——尤其是条件执行、移位寄存器和索引模式——通常仅见于CISC架构，但ARM将其纳入以缩减程序

因此，内存大小对于嵌入式设备和手持设备至关重要。然而，这些设计决策也导致了更复杂的硬件。

6.8 另一个视角：x86架构

如今几乎所有个人计算机都使用x86架构的微处理器。x86，也称为IA-32，是一种最初由英特尔开发的32位架构。AMD也销售兼容x86的微处理器。

x86架构历史悠久且复杂，可追溯至1978年英特尔发布16位8086微处理器之时。IBM为其首批个人电脑选用了8086及其衍生型号8088。1985年，英特尔推出了32位80386微处理器，该处理器向后兼容8086，因此能够运行早期PC开发的软件。与80386兼容的处理器架构被称为x86处理器。奔腾、酷睿和速龙处理器是广为人知的x86处理器。

多年来，英特尔和AMD的多个团队不断将更多指令和功能塞入过时的架构中。其结果远不如RISC-V优雅。然而，软件兼容性远比技术优雅重要，因此x86作为*de facto*个人电脑标准已超过二十年。每年有超过1亿颗x86处理器售出。这一巨大市场支撑着每年超过50亿美元的研发投入，以持续改进这些处理器。

x86是复杂指令集计算机（CISC）架构的一个例子。与RISC-V等RISC架构相比，每条CISC指令可以完成更多工作。CISC架构的程序通常需要更少的指令。指令编码被设计得更加紧凑，以在RAM比今天昂贵得多时节省内存；指令长度可变，通常小于32位。其代价是复杂指令更难解码，且执行速度往往更慢。

本节介绍x86架构。其目的并非将您培养成x86汇编语言程序员，而是为了说明x86与RISC-V之间的一些异同。我们认为了解x86的工作原理颇具趣味性。不过，理解本书其余内容并不需要掌握本节中的任何材料。x86与RISC-V（RV32I）的主要差异总结于表6.9。

6.8.1 x86 寄存器

8086微处理器提供了八个16位寄存器。它可以单独访问其中一些寄存器的高八位和低八位。当32位的80386推出时，这些寄存器被扩展到了

表6.9 RISC-V（RV32I）与x86的主要差异

Feature	RISC-V	x86
# of registers	32 general-purpose	8, some restrictions on purpose
# of operands	3 (2 sources, 1 destination)	2 (1 source, 1 source/destination)
Operand locations	Registers or immediates	Registers, immediates, or memory
Operand size	32 bits	8, 16, or 32 bits
Condition flags	No	Yes
Instruction types	Simple	Simple and complicated
Instruction encoding	Fixed: 4 bytes	Variable: 1–15 bytes

32位。这些寄存器被称为EAX、ECX、EDX、EBX、ESP、EBP、ESI和EDI。为了向后兼容，低16位以及部分低8位部分也可使用，如图6.36所示。

八个寄存器几乎（但不完全）是通用寄存器。某些指令不能使用某些寄存器，而其他指令则总是将结果存入特定寄存器。与RISC-V中的sp类似，ESP通常保留用作栈指针。

x86程序计数器被称为EIP（*extended instruction pointer*）。与RISC-V的PC类似，它从一条指令前进到下一条指令，或者可以通过分支和函数调用指令进行更改。

6.8.2 x86 操作数

RISC-V指令始终作用于寄存器或立即数。需要在内存和寄存器之间移动数据时，必须使用显式的加载和存储指令。相比之下，x86指令可以操作寄存器、立即数或内存。这在一定程度上弥补了寄存器数量较少的不足。

RISC-V指令通常指定三个操作数：两个源操作数和一个目标操作数。x86指令仅指定两个操作数。第一个是源操作数，第二个既是源操作数又是目标操作数。因此，x86指令总是用结果覆盖其一个源操作数。表6.10列出了x86中操作数位置的组合。除存储器到存储器外，所有组合都是可能的。

与RISC-V（RV32I）类似，x86拥有一个32位、按字节寻址的内存空间。然而，与RISC-V不同的是，x86支持更多种类的内存索引模式。内存位置可通过*base register*、*displacement*和*scaled index register*的任意组合来指定。表6.11展示了这些组合。位移量可以是一个

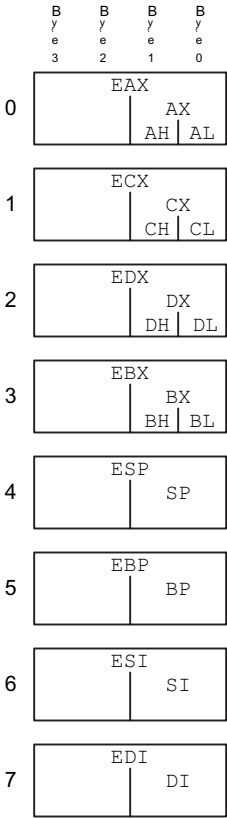


图 6.36 x86 寄存器

表6.10 操作数位置

Source/ Destination	Source	Example	Meaning
register	register	add EAX, EBX	EAX ← EAX + EBX
register	immediate	add EAX, 42	EAX ← EAX + 42
register	memory	add EAX, [20]	EAX ← EAX + Mem[20]
memory	register	add [20], EAX	Mem[20] ← Mem[20] + EAX
memory	immediate	add [20], 42	Mem[20] ← Mem[20] + 42

表6.11 内存寻址模式

Example	Meaning	Comment
add EAX, [20]	EAX ← EAX + Mem[20]	displacement
add EAX, [ESP]	EAX ← EAX + Mem[ESP]	base addressing
add EAX, [EDX+40]	EAX ← EAX + Mem[EDX+40]	base + displacement
add EAX, [60+EDI*4]	EAX ← EAX + Mem[60+EDI*4]	displacement + scaled index
add EAX, [EDX+80+EDI*2]	EAX ← EAX + Mem[EDX+80+EDI*2]	base + displacement + scaled index

表6.12 作用于8位、16位或32位数据的指令

Example	Meaning	Data Size
add AH, BL	AH ← AH + BL	8-bit
add AX, -1	AX ← AX + 0xFFFF	16-bit
add EAX, EDX	EAX ← EAX + EDX	32-bit

8位、16位或32位值。索引寄存器的缩放倍数可以是1、2、4或8。基址+位移模式等同于RISC-V加载和存储的基址寻址模式，但RISC-V指令不允许缩放。x86还提供了缩放索引。在x86中，缩放索引提供了一种便捷方式来访问2字节、4字节或8字节元素的数组或结构，而无需生成一系列指令来计算地址。虽然RISC-V始终对32位字进行操作，但x86指令可以对8位、16位或32位数据进行操作。表6.12展示了这些差异。

表 6.13 选定的 EFLAGS

Name	Meaning
CF (Carry Flag)	Carry out generated by last arithmetic operation. Indicates overflow in unsigned arithmetic. Also used for propagating the carry between words in multiple-precision arithmetic
ZF (Zero Flag)	Result of last operation was zero
SF (Sign Flag)	Result of last operation was negative (msb = 1)
OF (Overflow Flag)	Overflow of two's complement arithmetic

6.8.3 状态标志

x86与许多CISC架构一样，使用条件标志（也称为*status flags*）来决定分支并跟踪进位和算术溢出。x86使用一个名为EFLAGS的32位寄存器来存储状态标志。EFLAGS寄存器的一些位在表6.13中给出，其他位由操作系统使用。x86处理器的架构状态包括EFLAGS以及八个寄存器和EIP。

6.8.4 x86 指令

x86 的指令集比 RISC-V 更大。表 6.14 描述了一些通用指令。x86 还具有用于浮点运算以及对打包在较长字中的多个短数据元素进行运算的指令。D 表示目标（寄存器或内存位置），S 表示源（寄存器、内存位置或立即数）。

请注意，某些指令始终作用于特定寄存器。例如，32×32位乘法始终从EAX中获取一个源操作数，并将64位结果存入EDX和EAX。LOOP始终将循环计数器存储在ECX中。PUSH、POP、CALL和RET使用堆栈指针ESP。

条件跳转会检查标志位，并在满足相应条件时进行分支跳转。它们有多种形式。例如，JZ 在零标志位 (ZF) 为 1 时跳转，JNZ 在零标志位为 0 时跳转。这些跳转通常跟在设置标志位的指令（如比较指令 CMP）之后。表 6.15 列出了一些条件跳转及其如何依赖于先前比较操作所设置的标志位。与 RISC-V 不同，条件跳转（在 RISC-V 中称为条件分支）通常需要两条指令而非一条。

表6.14 选定的x86指令

Instruction	Meaning	Function
ADD/SUB	add/subtract	$D = D + S$ / $D = D - S$
ADDC	add with carry	$D = D + S + CF$
INC/DEC	increment/decrement	$D = D + 1$ / $D = D - 1$
CMP	compare	set flags based on $D - S$
NEG	negate	$D = -D$
AND/OR/XOR	logical AND/OR/XOR	$D = D \text{ op } S$
NOT	logical NOT	$D = \bar{D}$
IMUL/MUL	signed/unsigned multiply	$EDX:EAX = EAX \times D$
IDIV/DIV	signed/unsigned divide	$EDX:EAX/D$ $EAX = \text{quotient}; EDX = \text{remainder}$
SAR/SHR	arithmetic/logical shift right	$D = D \ggg S$ / $D = D \gg S$
SAL/SHL	left shift	$D = D \ll S$
ROR/ROL	rotate right/left	rotate D by S
RCR/RCL	rotate right/left with carry	rotate CF and D by S
BT	bit test	$CF = D[S]$ (the <i>S</i> th bit of D)
BTR/BTS	bit test and reset/set	$CF = D[S]; D[S] = 0 / 1$
TEST	set flags based on masked bits	set flags based on $D \text{ AND } S$
MOV	move	$D = S$
PUSH	push onto stack	$ESP = ESP - 4; \text{Mem}[ESP] = S$
POP	pop off stack	$D = \text{MEM}[ESP]; ESP = ESP + 4$
CLC, STC	clear/set carry flag	$CF = 0 / 1$
JMP	unconditional jump	relative jump: $EIP = EIP + S$ absolute jump: $EIP = S$
Jcc	conditional jump	if (flag) $EIP = EIP + S$
LOOP	loop	$ECX = ECX - 1$ if ($ECX \neq 0$) $EIP = EIP + \text{imm}$
CALL	function call	$ESP = ESP - 4;$ $\text{MEM}[ESP] = EIP; EIP = S$
RET	function return	$EIP = \text{MEM}[ESP]; ESP = ESP + 4$

表6.15 选定的分支条件

Instruction	Meaning	Function after <i>CMP D, S</i>
JZ/JE	jump if ZF = 1	jump if D = S
JNZ/JNE	jump if ZF = 0	jump if D ≠ S
JGE	jump if SF = 0F	jump if D ≥ S
JG	jump if SF = 0F and ZF = 0	jump if D > S
JLE	jump if SF ≠ 0F or ZF = 1	jump if D ≤ S
JL	jump if SF ≠ 0F	jump if D < S
JC/JB	jump if CF = 1	
JNC	jump if CF = 0	
JO	jump if OF = 1	
JNO	jump if OF = 0	
JS	jump if SF = 1	
JNS	jump if SF = 0	

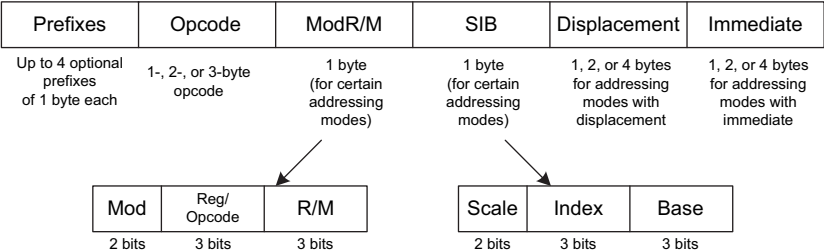


图 6.37 x86 指令编码

6.8.5 x86指令编码

x86指令编码确实非常混乱，这是数十年零散修改的遗留问题。与RISC-V不同（其指令统一为32位，压缩指令为16位），x86指令长度从1字节到15字节不等，如图6.37所示。³ *opcode*可能为1、2或3字节。其后跟随四个可选字段：*ModR/M*、*SIB*、*Displacement*和*Immediate*。*ModR/M*指定寻址模式。*SIB*指定特定寻址方式中的比例因子、索引寄存器和基址寄存器。

³It is possible to construct 17-byte instructions if all of the optional fields are used. However, x86 places a 15-byte limit on the length of legal instructions.

模式。*Displacement*表示某些寻址模式中的1、2或4字节位移。*Immediate*是使用立即数作为源操作数的指令中的1、2或4字节常量。此外，一条指令前最多可以添加四个可选的单字节前缀，以修改其行为。

*ModR/M*字节使用2位的*Mod*字段和3位的*R/M*字段来指定其中一个操作数的寻址模式。该操作数可以来自八个寄存器之一，也可以来自24种内存寻址模式之一。由于编码中的特殊问题，在某些寻址模式下，ESP和EBP寄存器不能用作基址或变址寄存器。*Reg*字段指定用作另一个操作数的寄存器。对于某些不需要第二个操作数的指令，*Reg*字段用于指定*opcode*的另外三个位。

在使用带比例索引寄存器的寻址模式时，*SIB*字节指定索引寄存器和比例（1、2、4或8）。如果同时使用基址和索引，则*SIB*字节还指定基址寄存器。

RISC-V 完全通过指令的 *op*、*funct3* 和 *funct7* 字段来指定指令。x86 使用可变数量的位来指定不同的指令，用更少的位表示更常见的指令，从而降低指令的平均长度。有些指令甚至拥有多个操作码。例如，ADD AL, imm8 执行一个 8 位立即数加法到 AL，其表示为 1 字节操作码 0x04 后跟 1 字节立即数。A 寄存器（AL、AX 或 EAX）被称为 *accumulator*。另一方面，ADD D, imm8 执行一个 8 位立即数加法到任意目标 D（内存或寄存器），其表示为 1 字节操作码 0x80，后跟一个或多个指定 D 的字节，再后跟 1 字节立即数。许多指令在目标为累加器时具有缩短的编码。

在最初的8086处理器中，操作码指定了指令是对8位还是16位操作数进行操作。当80386引入32位操作数时，没有新的操作码可用于指定32位形式。相反，16位和32位形式使用了相同的操作码。操作系统使用的 *code segment descriptor* 中的一个附加位指定了处理器应选择哪种形式。该位设置为0以向后兼容8086程序，默认操作码为16位操作数；设置为1则程序默认使用32位操作数。此外，程序员可以通过添加前缀来改变特定指令的形式。如果操作码前出现前缀0x66，则使用替代大小的操作数（在32位模式下为16位，在16位模式下为32位）。

6.8.6 其他x86特性

80286引入了 *segmentation*，将内存划分为长度不超过64 KB的段。当操作系统启用分段机制时，地址会

相对于段起始位置计算。处理器会检查超出段末尾的地址并指示错误，从而防止程序访问其自身段之外的内存。分段机制对程序员而言较为繁琐，因此在现代Windows操作系统版本中已不再使用。

x86包含对字节或字串进行操作的字符串指令。这些操作包括移动、比较或扫描特定值。在现代处理器中，这些指令通常比用一系列更简单的指令执行等效操作要慢，因此最好避免使用。

如前所述，0x66前缀用于选择16位或32位操作数大小。其他前缀包括用于锁定总线（以控制多处理器系统中对共享变量的访问）、预测分支是否会被执行以及在字符串移动期间重复指令的前缀。

英特尔与惠普在20世纪90年代中期联合开发了一种名为IA-64的新型64位架构。该架构从零开始设计，绕过了x86复杂的历史，充分利用了计算机架构领域20年的新研究成果，并提供了64位地址空间。然而，首款IA-64芯片上市过晚，未能取得商业成功。如今，大多数需要大地址空间的计算机都采用x86的64位扩展。

任何架构的致命弱点都是内存容量不足。使用32位地址时，x86可访问4 GB内存。这在1985年远超当时最大型计算机的容量。然而，到21世纪初's，这已变得捉襟见肘。2003年，AMD将地址空间和寄存器大小扩展至64位，并将增强后的架构命名为AMD64。AMD64具有兼容模式，可在不修改的情况下运行32位程序，同时操作系统可利用更大的地址空间。2004年，英特尔妥协并采纳了64位扩展，将其更名为扩展内存64技术（EM64T）。采用64位地址后，计算机可访问16艾字节（160亿GB）的内存。

对于有兴趣更详细研究x86架构的人，*x86 Intel Architecture Software Developer's Manual*可在英特尔网站上免费获取。

6.8.7 大局观

本节简要介绍了RISC-V架构与x86 CISC架构之间的一些差异。x86往往拥有更短的程序，因为一条复杂指令相当于一系列简单的RISC-V指令，并且其指令编码旨在最小化内存占用。然而，x86架构是多年来积累的各种功能的混合体，其中一些功能已不再有用，但为了与旧程序兼容而必须保留。

它的寄存器太少，且指令难以解码。仅解释指令集本身就很困难。尽管存在这些缺陷，x86 仍牢牢占据着个人电脑主导计算机架构的地位，因为软件兼容性的价值极为巨大，同时庞大的市场也使得研发高速 x86 微处理器的投入物有所值。

6.9 总结

要命令计算机，你必须说它的语言。计算机架构定义了如何命令处理器。如今，许多不同的计算机架构在商业上广泛使用，但一旦你理解了一种，学习其他架构就会容易得多。接触新架构时需要问的关键问题是：

数据字长是多少？► 寄存器有哪些？► 内存是如何组织的？► 指令有哪些？

RISC-V (RV32I) 是一种32位架构，因为它处理32位数据。RISC-V架构拥有32个通用寄存器。原则上，几乎任何寄存器都可用于任意目的。然而，为便于编程以及使不同程序员编写的函数能够轻松通信，按照惯例，某些寄存器被保留用于特定用途。例如，寄存器0（zero）始终保存常量0，ra保存jal指令后的返回地址，a0至a7保存函数的参数，a0至a1保存函数的返回值。RISC-V采用字节可寻址的内存系统，地址为32位。指令长度为32位，且为字对齐以实现高效访问。本章讨论了最常用的RISC-V指令。

定义计算机架构的优势在于，为某一特定架构编写的程序可以在该架构的多种不同实现上运行。例如，1993年为英特尔奔腾处理器编写的程序，通常仍能在2021年的英特尔i9或AMD锐龙处理器上运行（且运行速度更快）。

本书第一部分介绍了电路和逻辑抽象层次。本章中，我们跃升至架构层次。下一章将研究微架构，即实现处理器架构的数字构建模块的布局。

微架构是硬件与软件之间的桥梁。我们相信它是整个工程领域中最激动人心的课题之一：你将学会构建自己的微处理器！

练习

练习6.1 从RISC-V架构中分别给出以下架构设计原则的三个示例：(1) 规整性支持简洁性；(2) 加速常见情况；(3) 越小越快；(4) 优秀的设计需要良好的权衡。解释每个示例如何体现该设计原则。

练习6.2 RISC-V架构拥有一个由32个32位寄存器组成的寄存器组。是否可能设计一种没有寄存器组的计算机架构？如果可以，请简要描述该架构，包括指令集。这种架构相较于RISC-V架构有哪些优点和缺点？

练习6.3 使用ASCII编码编写以下字符串。用十六进制写出最终答案。

(a) 你好 (b) 一袋薯片 (c) 前来救援！

练习6.4 对以下字符串重复练习6.3。

(a) 酷 (b) RI
SC-V (c) 嘘
！

练习6.5 说明练习6.3中的字符串如何存储在从内存地址0x004F05BC开始的字节可寻址内存中。字符串的第一个字符存储在最低字节地址（本例中为0x004F05BC）。请清晰标明每个字节的内存地址。

练习6.6 对练习6.4中的字符串重复练习6.5。

练习6.7 nor指令不属于RISC-V指令集的一部分，因为相同的功能可以通过现有指令实现。编写一段简短的汇编代码片段，实现以下功能： $s3 = s4 \text{ NOR } s5$ 。尽可能使用最少的指令。

练习6.8 nand指令不属于RISC-V指令集，因为相同的功能可以通过现有指令实现。编写一段简短的汇编代码片段，实现以下功能： $s3 = s4 \text{ NAND } s5$ 。尽量使用最少的指令。

练习6.9 将以下高级代码片段转换为RISC-V汇编语言。假设（有符号）整数变量g和h分别位于寄存器a0和a1中。请为代码添加清晰的注释。

```
a) 如果 (g > h)
   则 g = g + 1; 否则
   h = h - 1;  b) 如果
   (g <= h)      则 g
   = 0; 否则      h
   = 0;
```

练习6.10 对以下代码片段重复练习6.9

ts.

```
a) 如果 (g >= h)
   则 g = g + h;
   否则 g = g - h;  b)
   如果 (g < h)
   则 h = h + 1;
   否则 h = h * 2;
```

练习6.11 将以下高级代码片段转换为RISC-V汇编。假设array1和array2的基地址分别保存在t1和t2中，且array2数组在使用前已初始化。尽量使用最少的指令。请清晰注释你的代码。

```
int i; int array1[100]; int array2[100]; ... for (i
= 0; i < 100; i = i + 1) array1[i] = array2[i];
```

练习6.12 对以下高级代码片段重复练习6.11。假设temp数组在使用前已初始化，且t3保存temp的基地址。

```
int i; int temp[100]; ... for (i = 0; i < 100; i = i
+ 1) temp[i] = temp[i] * 128;
```

练习6.13 编写RISC-V汇编代码，将以下立即数（常量）存入s7。使用最少数量的指令。

- (a) 29 (b) -214 (c) -2999 (d) 0xABCDE000 (e) 0xEDCBA123 (f) 0xEEEEFEFA
B

练习6.14 对以下立即数重复练习6.13。

- (a) 47 (b) -349 (c) 5328 (d) 0xBBCCD000 (e) 0xFEEBC789 (f) 0xCCAAB9AB

练习6.15 用高级语言编写一个函数 `int find42(int array[], int size)`。size 指定数组中的元素个数，`array[]` 指定数组的基地址。该函数应返回第一个值为42的数组元素的索引号。如果没有数组元素为42，则应返回值 -1。请为代码添加清晰的注释。

练习6.16 高级函数strcpy（字符串复制）将字符串src复制到字符串dst。

```
// C代码
void strcpy(char dst[], char src[]) {
    int i = 0;
    do { dst[i] = src[i]; } while (src[i++] != '\0');
}
```

- (a) 在RISC-V汇编代码中实现strcpy函数。使用t0作为i。 (b) 画出strcpy函数调用前、调用期间和调用后的栈图。假设在调用strcpy之前，`sp = 0xFFC000`。

练习6.17 将练习6.15中的高级函数转换为RISC-V汇编代码。请清晰注释你的代码。

这个简单的字符串复制函数存在一个严重缺陷：它无法知道目标缓冲区是否有足够空间来接收源字符串。如果恶意程序员能够用长字符串作为源来执行strcpy，就可能在整个内存中写入字节，甚至可能修改后续内存位置中存储的代码。通过一些巧妙手段，被修改的代码可能会接管机器。这被称为**buffer overflow**；它被多种恶意程序利用，包括臭名昭著的冲击波蠕虫，该蠕虫在2003年造成了约5.25亿美元的损失。

练习6.18 考虑下面的RISC-V汇编代码。func1、func2和func3是非叶子函数。func4是一个叶子函数。每个函数的代码未显示，但注释指出了每个函数内部使用的寄存器。你可以假设这些函数不需要在它们的栈上保存任何非保留寄存器。

```
0x00091000 func1: ... # func1 使用 t2–t3, s4–s10 ... 0x00091020    jal f
unc2 ... 0x00091100 func2: ... # func2 使用 a0–a2, s0–s5 ... 0x0009117C
jal func3 ... 0x00091400 func3: ... # func3 使用 t3, s7–s9 ... 0x00091704
jal func4 ... 0x00093008 func4: ... # func4 使用 s10–s12 ... 0x00093118
jr ra
```

- (a) 每个函数的栈帧有多少个字？
- (b) 画出调用 func4 后的栈结构。清晰标明哪些寄存器存储在栈上的哪个位置，并标记每个栈帧。尽可能给出具体值。假设在调用 func1 之前，sp = 的值为 0xA BC124。

练习6.19 斐波那契数列中的每个数字都是前两个数字之和。表6.16列出了该数列的前几个数字，*fib*(*n*)。

- (a) 对于 *n* = 0 和 *n* = −1，*fib*(*n*) 是什么？
- (b) 用高级语言编写一个名为 fib 的函数，该函数返回任意非负整数 *n* 对应的斐波那契数。提示：你可能需要使用循环。请为代码添加清晰的注释。
- (c) 将第(b)部分的高级函数转换为RISC-V汇编代码。在每行代码后添加注释，清晰解释其功能。使用模拟器在*fib*(9)上测试你的代码。（关于RISC-V模拟器的链接请参见前言。）

表6.16 斐波那契数列

<i>n</i>	1	2	3	4	5	6	7	8	9	10	11	...
<i>fib</i> (<i>n</i>)	1	1	2	3	5	8	13	21	34	55	89	...

练习6.20 考虑代码示例6.28。在本练习中，假设 $factorial(n)$ 被调用时输入参数为 $n = 5$ 。

- (a) 当 $factorial$ 返回到调用函数时， $a0$ 中的值是多少？
- (b) 假设你将地址 $0x8508$ 和 $0x852C$ 处的指令替换为 nop 指令。程序是否会：
- (1) 进入无限循环但不崩溃；
 - (2) 崩溃（导致栈增长或收缩超出动态数据段，或使程序计数器跳转到程序外的位置）；
 - (3) 当程序返回循环时，在 $a0$ 中产生一个错误的值（如果是，什么值？）；或
 - (4) 尽管删除了行，仍能正确运行？
- (c) 重复(b)部分，但需对指令进行以下修改：
- (1) 将地址 $0x8504$ 和 $0x8528$ 处的指令替换为 nop 。
 - (2) 将地址 $0x8518$ 处的指令替换为 nop 。
 - (3) 将地址 $0x8530$ 处的指令替换为 nop 。

练习6.21 Ben Bitdiddle 试图计算非负整数 b 的函数 $f(a, b) = 2a + 3b$ 。他过度使用了函数调用和递归，并为函数 f 和 g 生成了以下高级代码。

```
// 函数 f 和 g 的高级代码
int f(int a, int b) { int j; j = a;
return j + a + g(b); }
int g(int x) { int k; k = 3; if (x ==
0) return 0; else return k + g(x - 1); }
```

Ben 随后将这两个函数翻译为 RISC-V 汇编语言如下。他还编写了一个函数 $test$ ，用于调用函数 $f(5, 3)$ 。

```
# RISC-V 汇编代码 # f: a0 = a, a1 = b, s4 = j; # g: a0 = x, s4 = k
0x8000 test:
addi a0, zero, 5 # a = 5
0x8004 addi a1, zero, 3 # b = 3
0x8008 jal f # 调用 f(5, 3)
0x800C loop: j loop # 无限循环
0x8010 f: addi sp, sp, -16 # 在栈上分配空间
0x8014 sw a0, 0xC(sp) # 保存 a0
0x8018 sw a1, 0x8(sp) # 保存 a1
```

```

0x801C sw ra, 0x4(sp) # 保存 ra 0x8020 sw s4, 0x0(sp) # 保存 s4 0x8024 addi s
4, a0, 0 # j = a 0x8028 addi a0, a1, 0 # 将 b 作为参数传递给 g() 0x802C jal g #
调用 g 0x8030 lw t0, 0xC(sp) # 将 a 恢复到 t0 0x8034 add a0, a0, t0 # a0 = g(b)
+ a 0x8038 add a0, a0, s4 # a0 = (g(b) + a) + j 0x803C lw s4, 0x0(sp) # 恢复寄存
器 0x8040 lw ra, 0x4(sp) 0x8044 addi sp, sp, 16 0x8048 jr ra # 返回 0x804C g: a
ddi sp, sp, -8 # 在栈上分配空间 0x8050 sw ra, 4(sp) # 保存寄存器 0x8054 sw s4,
0(sp) 0x8058 addi s4, zero, 3 # k = 3 0x805C bne a0, zero, else # 如果 (x != 0), 跳
转到 else 0x8060 addi a0, zero, 0 # 返回 0 0x8064 j done # 清理并返回 0x8068 el
se: addi a0, a0, -1 # 递减 x 0x806C jal g # 调用 g(x - 1) 0x8070 add a0, s4, a0
# 返回 k + g(x - 1) 0x8074 done: lw s4, 0(sp) # 恢复寄存器 0x8078 lw ra, 4(sp) 0x
807C addi sp, sp, 8 0x8080 jr ra # 返回

```

你可能会发现，绘制类似于图6.10中的栈图有助于回答以下问题。

(a) 如果代码从test开始运行，当程序执行到loop时，a0中的值是多少？该程序是否正确计算了 $2a + 3b$ ？

(b) 假设Ben将地址0x8014处的指令替换为nop。程序是否

- (1) 进入无限循环但不崩溃；
- (2) 崩溃（导致栈增长或收缩超出动态数据段，或使程序计数器跳转到程序外的位置）；
- (3) 当程序返回循环时，在a0中产生一个错误的值（如果是，什么值？）；或
- (4) 尽管删除了行，仍能正确运行？

(c) 当以下指令被更改时，重复部分(b)。请注意，标签未更改，仅指令被更改。

- (i) 0x8014和0x8030处的指令被替换为nops。
- (ii) 0x803C和0x8040处的指令被替换为nops。
- (iii) 0x803C处的指令被替换为nop。
- (iv) 0x8030处的指令被替换为nop。
- (v) 0x8054和0x8074处的指令被替换为nops。

(vi) 地址0x8020和0x803C处的指令被替换为nop。

(vii) 地址0x8050和0x8078处的指令被替换为nop。

练习6.22 将以下RISC-V汇编代码转换为机器语言。用十六进制写出指令。

```
addi s3, s4, 28 sll t1,
t2, t3 srli s3, s1, 14 s
w s9, 16(t4)
```

练习6.23 对以下RISC-V汇编代码重复练习6.22:

```
add s7, s8, s9 srai t0, t1,
0xC ori s3, s1, 0xABC l
w s4, 0x5C(t3)
```

练习6.24 考虑所有带有立即数字段的指令。

(a) 习题6.22中的哪些指令在其机器码格式中使用了立即数字段? (b) 第(a)部分中的指令属于哪种指令类型 (I型、S型、B型、U型或J型)?

(c) 写出(a)部分每条指令的5至21位立即数的十六进制表示。如果该数被扩展, 还需写出其32位扩展后的立即数。否则, 注明其未被扩展。

练习6.25 对练习6.23中的指令重复练习6.24。

练习6.26 考虑下面的RISC-V机器代码片段。第一条指令列在顶部。

(a) 将机器代码片段转换为RISC-V汇编语言。 (b) 逆向工程一个能够编译成该汇编语言例程的高级程序, 并编写该程序。请清晰地代码添加注释。

(c) 用文字解释该程序的功能。a0 和 a1 是输入, 初始时包含正数 A 和 B。程序结束时, 寄存器 a0 保存输出 (即返回值)。

```
0x01800513 0
x00300593 0x
00000393 0x0
0058E33 0x01
C54863 0x001
38393 0x00BE
0E33 0xFF5FF
06F 0x000385
33
```

练习6.27 对以下机器码重复练习6.26。a0和a1是输入。a0包含一个32位数，a1是一个32元素字符数组的地址。

```
0x01F00393 0
x00755E33 0x
001E7E13 0x0
1C580A3 0x0
0158593 0xFF
F38393 0xFE0
3D6E3 0x0000
8067
```

练习6.28 将下列分支指令转换为机器码。每条指令的地址已标注在左侧。

```
a) 0x0000A000 beq t4, zero, Loop 0x0000A004 ...
0x0000A008 ... 0x0000A00C Loop: ... b) 0x00801
000 bne s5, a1, L1 ... 0x0080174C L1: ... c) 0x0
000C10C Back: ... 0x0000D000 blt s1, s2, Bac
k d) 0x01030AAC bge t4, t6, L2 ... 0x01031AA
4 L2: ... e) 0x0BC08004 L3: ... 0x0BC09000
beq s3, s7, L3
```

练习6.29 将下列分支指令转换为机器码。每条指令的地址已标注在左侧。

```
a) 0xAA00E124 blt t4, s3, Loop 0xAA00E128 ...
0xAA00E12C ... 0xAA00E130 Loop: ... b) 0xC0
901000 bge t1, t2, L1 ... 0xC090174C L1: ...
```

c) 0x1230D10C 返回: 0x1230D908 bne s10
 , s11, 返回 d) 0xAB0C99A8 beq a0, s1, L2 0
 xAB0CA0FC L2: ... e) 0xFFABCF04 L3: 0
 xFFABD640 blt s1, t3, L3

练习6.30 将以下跳转指令转换为机器码。每条指令的地址已标注在指令左侧。

a) 0x1234ABC0 j Loop 0x123CABB
 C Loop: ... b) 0x12345678 Back: 0x
 123B8760 jal s0, Back c) 0xAABBCCD0 j
 al L1 0xAABDCD98 L1: ... d) 0x1122
 3344 j L2 0x1127BCDC L2: ... e) 0x9
 876543C L3: 0x9886543C jal L3

练习6.31 将下列跳转指令转换为机器码。每条指令的地址已标注在指令左侧。

a) 0x0000ABC0 jal Loop 0x0000E
 EEC Loop: ... b) 0x0000C10C Back:
 0x000F1230 jal Back c) 0x0080100
 0 jal s1, L1 0x008FFFDC L1: ...

d) 0xA1234560 j L2 0xA13
 1347C L2: ... e) 0xF0BBCCD4
 L3: 0xF0CBCCD4 j L3

练习6.32 考虑以下RISC-V汇编语言片段。每条指令左侧的数字表示指令地址。

```
0xA0028 Func1: addi t4, a1, 0 0xA002C ori
a0, a0, 32 0xA0030 sub a1, a1, a0 0xA0034
jal Func2 ... .. 0xA0058 Func2: lw t2, 4(a0)
0xA005C sw t2, 16(a1) 0xA0060 srli t3, t2,
8 0xA0064 beq t2, t3, Else 0xA0068 jr ra
0xA006C Else: addi a0, a0, 4 0xA0070 j Fu
nc2
```

(a) 将指令序列转换为机器码。以十六进制形式写出机器码指令。

(b) 列出每行代码使用的指令类型和寻址模式。

练习6.33 考虑以下C代码片段。

```
// C代码 void setArray(int num) { int i; i
nt array[10]; for (i = 0; i < 10; i = i + 1) ar
ray[i] = compare(num, i); } int compare(int a,
int b) { if (sub(a, b) >= 0) return 1; else
return 0; } int sub(int a, int b) { return a - b
; }
```

(a) 将C代码片段实现为RISC-V汇编语言。使用s4保存变量i。务必适当处理栈指针。数组存储在setArray函数的栈上（参见第6.3.7节末尾）。请为代码添加清晰的注释。

(b) 假设 `setArray` 是第一个被调用的函数。绘制调用 `setArray` 之前以及每次函数调用期间栈的状态。标明栈地址、存储在栈上的寄存器和变量名称；标记 `sp` 的位置；并清晰标出每个栈帧。假设 `sp` 起始地址为 `0x8000`。

(c) 如果未能将 `ra` 存储在栈上，你的代码将如何运行？

练习6.34 考虑以下高级函数。

```
// C代码
int f(int n, int k) {
    int b;
    b = k + 2;
    if (n == 0)
        b = 10;
    else
        b = b + (n * n) + f(n - 1, k + 1);
    return b * k;
}
```

(a) 将高级函数 `f` 翻译为 RISC-V 汇编语言。特别注意在函数调用过程中正确保存和恢复寄存器，并遵循 RISC-V 保留寄存器约定。请清晰注释代码。假设函数起始指令地址为 `0x8100`。将局部变量 `b` 保存在 `s4` 中。请清晰注释代码。

(b) 手动逐步执行你在(a)部分编写的函数，针对 `f(2, 4)` 的情况。绘制一个类似于图6.10的堆栈图，假设当 `f` 被调用时 `sp` 等于 `0xBFF00100`。写出堆栈地址以及每个位置存储的寄存器名称和数据值，并跟踪堆栈指针值(`sp`)。清晰标记每个堆栈帧。你可能会发现跟踪 `a0`、`a1` 和 `s4` 在执行过程中的值也很有用。假设当 `f` 被调用时，`s4 = 0xABCD`，`ra = 0x8010`。

(c) 当调用 `f(2, 4)` 时，`a0` 的最终值是多少？

练习6.35 一条分支指令（如 `beq`）最多可以向前分支（即跳转到更高的指令地址）多少条指令？

练习6.36 一条分支指令（如 `beq`）向后分支（即指向较低指令地址）的最大指令数是多少？

练习6.37 编写汇编代码，使其从给定指令开始有条件地跳转到向前32兆指令的位置。请注意，1兆指令 = 2^{20} 指令 = 1,048,576条指令。假设你的代码从地址 `0x8000` 开始。使用最少数量的指令。

练习6.38 解释为什么在跳转并链接指令jal的机器格式中拥有一个大的立即数字段是有利的。

练习6.39 考虑一个函数，它接收一个包含10个32位整数的小端格式数组，并将其转换为大端格式。

(a) 用高级代码编写此函数。

(b) 将该函数转换为RISC-V汇编代码。注释所有代码并使用最少数量的指令。

练习6.40 考虑两个字符串：string1和string2。

(a) 编写一个名为 concat 的函数的高级代码，该函数用于连接两个字符串：void concat(char string1[], char string2[], char stringconcat[])。该函数不返回任何值。它将 string1 和 string2 连接起来，并将结果字符串放入 stringconcat 中。你可以假设字符数组 stringconcat 足够大，能够容纳连接后的字符串。请为代码添加清晰的注释。(b) 将 (a) 部分中的函数转换为 RISC-V 汇编语言。请为代码添加清晰的注释。

练习6.41 编写一个RISC-V汇编程序，将保存在a0和a1中的两个正单精度浮点数相加。不得使用任何RISC-V浮点指令。无需担心任何为特殊用途保留的编码（例如0、NaN等）或溢出/下溢的数字。使用模拟器测试你的代码。（有关RISC-V模拟器的链接，请参阅前言。）你需要手动设置a0和a1的值来测试代码。证明你的代码能够可靠运行。请为代码添加清晰的注释。

练习6.42 将练习6.41中的RISC-V汇编程序扩展为能够处理正负单精度浮点数。请清晰注释你的代码。

练习6.43 考虑一个将名为scores的10元素数组从低到高排序的函数。函数完成后，scores[0]保存最小值，scores[9]保存最大值。

(a) 编写一个高级排序函数，实现上述功能。sort 接收一个参数，即 scores 数组的地址。请清晰注释你的代码。

(b) 将排序函数转换为RISC-V汇编语言。请清晰地代码添加注释。

练习6.44 考虑以下RISC-V程序。假设指令从内存地址0x8400开始存放，全局变量x和y分别位于内存地址0x10024和0x10028。

```
# RISC-V 汇编代码 main:  addi sp, sp, -4 # 在栈上分配空间
sw ra, 0(sp) # 将 ra 保存到栈上 lw a0, -940(gp) # a0 = x
lw a1, -936(gp) # a1 = y jal diff
# 调用 diff() lw ra, 0(sp) # 恢复寄存器 addi sp, sp, 4
jr ra # 返回 diff: sub a0, a0, a1 # 返回 (a0-a1) jr ra
```

- (a) 首先，在每个汇编指令旁边显示指令地址。
- (b) 描述符号表：即列出每个符号（即函数标签和全局变量）的名称、地址和大小。
- (c) 将所有指令转换为机器码。
- (d) 数据段和文本段有多大（多少字节）？
- (e) 绘制一个内存映射图，显示数据和指令的存储位置，类似于图6.34。请确保在程序开始时标注PC和gp的值。

练习6.45 对以下RISC-V代码重复练习6.44。假设指令从内存地址0x8534开始存放，全局变量g和h分别位于内存地址0x1305C和0x13060。

```
# RISC-V 汇编代码 main:  addi sp, sp, -8 sw ra, 4(sp) sw s4, 0(sp)
addi s4, zero, 15 sw s4, -300(gp) # g = 15 addi a1, zero, 27 # arg1 = 27
sw a1, -296(gp) # h = 27 lw a0, -300(gp) # arg0 = g = 15
jal greater lw s4, 0(sp) lw ra, 4(sp) addi sp, sp, 8
jr ra greater: blt a1, a0, isGreater addi a0, zero, 0
jr ra isGreater: addi a0, zero, 1 jr ra
```

练习6.46 解释RISC-V立即数编码中位交错（bit-swizzling）的优缺点。

练习6.47 考虑RISC-V指令中立即数的符号扩展。按照以下步骤设计一个用于RISC-V立即数的符号扩展单元。最小化所需的硬件量。

- (a) 绘制一个符号扩展单元的示意图，该单元对 I 型指令中的 12 位立即数进行符号扩展。电路的输入是指令的高 12 位 $Instr_{31:20}$ ，它编码了 12 位有符号立即数。输出是一个 32 位符号扩展后的立即数 $ImmExt_{31:0}$ 。
- (b) 将(a)部分中的符号扩展单元扩展，使其也能对S型指令中的12位立即数进行符号扩展。根据需要修改输入，并在可能的情况下重用硬件。
- (c) 将(b)部分中的符号扩展单元扩展，使其也能对B型指令中的13位立即数进行符号扩展。
- (d) 将(c)部分中的符号扩展单元扩展，使其也能对J型指令中的21位立即数进行符号扩展。

练习6.48 在本练习中，你将使用最少的硬件为RISC-V立即数设计一个替代扩展单元。假设RISC-V架构师选择使用对人类更直观的立即数编码，如图6.38所示。该图展示了除操作码外的所有指令字段。图6.38中的立即数编码未使用位交错（但仍将S/B类指令中的立即数字段拆分为两个指令字段）。与真实RISC-V立即数编码（如图6.27所示）不同的位以蓝色高亮显示。具体而言，这些假设的（直观）立即数编码与真实RISC-V立即数编码在B类和J类格式上存在差异。

- (a) 绘制一个符号扩展单元的示意图，该单元对I型指令中的12位立即数进行符号扩展。电路的输入是

11 10 9 8 7 6 5 4 3 2 1 0	rs1	funct3	rd	I
11 10 9 8 7 6 5	rs2	rs1	funct3 4 3 2 1 0	S
12 11 10 9 8 7 6	rs2	rs1	funct3 5 4 3 2 1	B
31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12	rd			U
20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1	rd			J

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7

图6.38 替代立即数编码

指令 $Instr_{31:20}$ ，它编码了12位有符号立即数。输出是一个32位符号扩展的立即数 $ImmExt_{31:0}$ 。

(b) 将(a)部分中的符号扩展单元扩展，使其也能对S型指令中的12位立即数进行符号扩展。根据需要修改输入，并在可能的情况下重用硬件。

(c) 将(b)部分中的符号扩展单元扩展，使其也能对（修改后的）B型指令中的13位立即数进行符号扩展（见图6.38）。

(d) 将(c)部分中的符号扩展单元扩展，使其也能对（修改后的）J型指令中的21位立即数进行符号扩展（见图6.38）。(e) 如果你完成了练习6.47，请将其与实际RISC-V扩展单元中的硬件进行比较。

练习6.49 考虑jal指令可以跳转多远。

(a) jal指令可以向前跳转（即跳转到更高地址）多少条指令？(b) jal指令可以向后跳转（即跳转到更低地址）多少条指令？

地址)？

练习6.50 考虑在一个字节可寻址的内存中，存储在第42nd个内存字上的一个32位字的存储。请记住，第0th个字存储在内存地址0处，第一个字存储在地址4处，依此类推。

(a) 存储在内存中的第42nd个字的字节地址是什么？

(b) 42nd字跨越的字节地址是什么？

(c) 绘制存储在字42处的数值0xFF223344在大端序和小端序机器中的存储方式。清晰标注每个数据字节值对应的字节地址。

练习6.51 对于存储在字节可寻址存储器中第15th个字的32位字的存储，重复练习6.50。

练习6.52 解释以下RISC-V程序如何用于判断计算机是大端序还是小端序：

```
addi s7, 100 lui s3, 0xABCD8 # s3 = 0xABCD8000
addi s3, s3, 0x765 # s3 = 0xABCD8765 sw s3, 0(s7)
lb s2, 1(s7)
```

面试问题

以下练习题展示了数字设计岗位面试中曾出现的问题。

问题6.1 编写一个RISC-V汇编语言程序，用于交换两个寄存器a0和a1的内容。你不能使用任何其他寄存器。

问题 6.2 假设给定一个包含正负整数的数组。编写RISC-V汇编代码，找出该数组中具有最大和的子集。假设数组的基地址和数组元素数量分别存储在a0和a1中。你的代码应将得到的子集从基地址a2开始存放。编写尽可能快速运行的代码。

问题6.3 给定一个以空字符结尾的字符串数组，其中包含一个句子。编写一个RISC-V汇编语言程序，将句子中的单词顺序反转，并将新句子存回该数组。

问题6.4 编写一个RISC-V汇编语言程序，统计一个32位数字中1的个数。

问题6.5 编写一个RISC-V汇编语言程序，反转寄存器中的位。使用尽可能少的指令。

问题6.6 编写一个简洁的RISC-V汇编语言程序，测试从a3中减去a2时是否发生溢出。

问题6.7 编写一个RISC-V汇编语言程序，用于测试给定字符串是否为回文。（回顾：回文是指正读反读都相同的单词。例如，“wow”和“racecar”都是回文。）

